04820KA001 Mono Slitter 903_V12_1112_1

Author: Michou
Created: 11/12/2026 8:52:20 AM
Last Modified: 11/12/2026 1:54:57 PM
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

Table of Contents

04820KA001 Mono Slitter 903_V12_1112_1	3
1.PLC	3
1-8.POUs	3
1-8-1.Programs	3
1-8-1-1.TaglMonoTRM	3
1-8-1-1-1.Variables	3
1-8-1-1-2.General	22
1-8-1-1-3.Motor_Manager	28
1-8-1-1-4.OverRider_Manager	45
1-8-1-1-5.Console	48
1-8-1-1-6.Alarms	53
1-8-1-1-7.Output_Manager	60
1-8-1-1-8.AutoStop_Control	65
1-8-1-1-9.Tension_Manager	89
1-9.Data	93
1-9-1.Data Types	93
1-9-2.Global Variables	94

1.PLC

1-8.POUs

1-8-1.Programs

1-8-1-1.TaglMonoTRM

1-8-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
UwSlowDiamCalc	REAL			False	False	V7c: speed-dependent unwinder slow-down diameter (mm)
SonarRunMem	bool			False	False	
UwDiamAtLatch	REAL			False	False	Unwinder diameter when the slow-down point was latched (release check)
EndStopReq	BOOL			False	False	V12: end-of-roll stop requested (AutoStopCmd or unwinder diameter)
Buffer_R	BOOL			False	False	Buffer Result
EndRampOK	BOOL			False	False	V12: end-of-roll ramp usable (FinalStopDecel, FF active, running)
EndRampHard	BOOL			False	False	V12: safety net during the ramp -> immediate stop
EndStopNow	BOOL			False	False	V12: immediate end-of-roll stop (Motor_Manager rung 4)
TensTarget	REAL			False	False	Tension Manager: target before rate limit (kg)
TensStep	REAL			False	False	Tension Manager: max change per 4 ms scan (kg)
TensAOReal	REAL			False	False	Tension Manager: AO counts before INT conversion
SonarInit	bool			False	False	
AlarmMem	ARRAY[0..32] OF bool			False	False	
UwSlowDiamLatch	BOOL			False	False	V7c: unwinder diameter slow-down reached (frozen)
FFRate	REAL			False	False	V7f: ramp rate (reference per 4 ms scan)
FFRateSet	REAL			False	False	V7f: rate allowed to still stop at the target

FFJerk	REAL			False	False	V7f: max rate change per scan
SpdScanCnt	INT			False	False	V9: scan counter for the 100 ms speed window
TensGSMMem	REAL			False	False	
TensWidthMem	REAL			False	False	
TensKraftMem	BOOL			False	False	
HornRun	BOOL			False	False	V9: slow-down / final speed running
HornRunMem	BOOL			False	False	V9: edge memory
HornStartMem	BOOL			False	False	V9: start edge memory
HornTicks	INT			False	False	V9: scans since slow-down start
HornCnt	INT			False	False	V9: scans in the beep period
HornPer	INT			False	False	V9: beep period in scans
HornStartScans	INT			False	False	V9: start beep in scans
HornLenScans	INT			False	False	V9: short beep in scans
HornSlowBeep	BOOL			False	False	V9: slow-down beep
HornStopPend	BOOL			False	False	V9: stopped after a slow-down, waiting for standstill
HornStopTicks	INT			False	False	V9: stop beep counter
LongActive	BOOL			False	False	V9: long-baseline learning running
LongValid	BOOL			False	False	V9: long baseline >= 20 mm growth, replaces the windows
LongStartPos	DINT			False	False	V9: encoder at the start of the long baseline
LongStartD	REAL			False	False	V9: rewinder diameter at the start (mm)
LongGrow_mm	REAL			False	False	V9: growth since the start (mm)
LongPls	REAL			False	False	V9: pulses since the start
LongK	REAL			False	False	V9: K from the long baseline
RevW	REAL			False	False	V9: encoder pulses summed in the current revolution

RevDP	DINT			False	False	V9: encoder pulses this scan
RevPrevPos	DINT			False	False	V9: encoder last scan
RevSum	REAL			False	False	V9: sum of raw DT35 distance x pulses
RevMin	REAL			False	False	V9: min raw distance in the revolution
RevMax	REAL			False	False	V9: max raw distance in the revolution
RwDiamRevOk	BOOL			False	False	V9: one full revolution averaged since the last fault
UwEmptyTicks	INT			False	False	V9: unwinder empty time (100 ms ticks)
UwFullTicks	INT			False	False	V9: new roll seen time (100 ms ticks)
GsmArmed	BOOL			False	False	V9: roll was empty -> next new roll asks the GSM question
T_JogStartBlk	TOF			False	False	V9: START blocked while JOG held + 2 s
StartHeldBlk	BOOL			False	False	V9: START pressed during the block, wait for release
StartPB_OK	BOOL			False	False	V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
FFSoftMem	BOOL			False	False	V9: soft stop last scan (rung B)
FFSlowAcc	BOOL			False	False	V9: gentle ramp-up after a cancelled soft stop
FFStartEdge	BOOL			False	False	V9: START push-button edge (rung B)
FFStartMem	BOOL			False	False	V9: START last scan (rung B)
SpdDeltaMin	REAL			False	False	Minimum speed difference
FFSqrtArg	REAL			False	False	SQRT argument (REAL)
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
SoftStopMem	BOOL			False	False	v6r: soft stop edge memory
SoftStopRatio	REAL			False	False	v6r: InvRef counts per m/min, frozen at the soft stop start
MemPos	DINT			False	False	Previous encoder count
MatChangeCnt	INT			False	False	v6s: consecutive learning windows far from RwMat

						_K
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running
TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas_mm	REAL			False	False	Measured slow-down decel (mm/s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarms
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command

HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Cooling Fan command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Hoover	BOOL			False	False	OverRider in Hoover Mode
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
RwAutoLimAct	BOOL			False	False	v6y: rewinder AUTO speed limit active (taken over at standstill)
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Acti	BOOL			False	False	Roll diameter photocell int

ve						ercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS
UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised

LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
StopLen_Pls	REAL			False	False	Stop length (pulses)
StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
T_ByPassProtOff	TOF			False	False	Delay Off Timer

UwEnMem	BOOL			False	False	Edge memory UwStopEnable
RwEnMem	BOOL			False	False	Edge memory RwEnable
SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
SqrtArg	REAL			False	False	Square root argument
AutoSlowCmd	BOOL			False	False	Automatic Slow Command
AutoStopCmd	BOOL			False	False	Automatic Stop Command
RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
UnWinder_Empty	BOOL			False	False	UnWinder Empty
StopFailCnt	INT			False	False	Consecutive failed stop measurements
UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
T_MatReset	TON			False	False	v6s: RwMatReset stays ON 3 s
RwAutoMaxSpdAct	REAL			False	False	v6y: active rewinder AUTO max speed (taken over at standstill)
UwEmptyMem	BOOL			False	False	v6x: UnWinder_Empty edge memory
LearnWinLen_Pls	REAL			False	False	v6w: learning window length in encoder pulses
WebRatio	REAL			False	False	V7: drive counts per m/min of web at the current diameter
FFActive	BOOL			False	False	V7: feed-forward loop active
FFWasActive	BOOL			False	False	V7: edge memory
FFTarget	REAL			False	False	V7: reference for the set speed (mm/s of the 0-10 V scale)
RampRef	REAL			False	False	V7: ramped reference (mm/s of the 0-10 V scale)

StepAcc	REAL			False	False	V7: accel step per scan
StepDec	REAL			False	False	V7: decel step per scan
FFSteadyTicks	INT			False	False	V7: scans at target
FFErr	REAL			False	False	V7: relative speed error
Clock_100ms	bool			False	False	Clock100ms
AlarmCnt	INT			False	False	
AlarmCntMem	INT			False	False	
RstEdge	BOOL			False	False	
RstMem	BOOL			False	False	
SilenceTicks	INT			False	False	
WarnCnt	INT			False	False	
WarnCntMem	INT			False	False	
WarnBeepTicks	INT			False	False	
TensUnitMem	BOOL			False	False	
TensIdeal_N	REAL			False	False	
TensionSeen	BOOL	False		False	False	Tension reached once since START
T_TensOn	TON			False	False	Tension ON debounce 0.5 s
T_TensOff	TON			False	False	Tension OFF debounce 2 s
T_UwSensFlt	TON			False	False	Unwinder sonar fault debounce 1 s
Buffer_A	BOOL			False	False	
Buffer_B	BOOL			False	False	
Buffer_C	BOOL			False	False	
Buffer_D	BOOL			False	False	
AS_RunMem	BOOL			False	False	
AutoStopSeen	BOOL			False	False	
T_AutoStopDisp	TON			False	False	
RwArmMem	BOOL			False	False	
UwArmMem	BOOL			False	False	
T_AutoEnRef	TON			False	False	
TensRef	REAL			False	False	
WebBreakNow	BOOL			False	False	
CntUwWasEmpty	bool			False	False	
T_CntNewRoll	ton			False	False	
CntPulseReq	BOOL			False	False	

	CntPulseDone	BOOL			False	False	Metre reset: pulse given, waiting for counter ~0
	CntStartMem	BOOL			False	False	
	T_CntResetPulse	tp			False	False	
	T_CntDone	ton			False	False	
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percent	REAL				False	Speed difference percentage
	Encoder_Pos	DINT				False	Encoder counter
	FinalStopDecel	REAL				False	
	FinalStopRamp	BOOL				False	
	Velocity_M_min	REAL				False	Speed in m/min
	Current_Metre	REAL				False	Current metres
	PaperKraft	BOOL				False	
	TensFactorBleached	REAL				False	
	TensFactorKraft	REAL				False	
	TensAuto	BOOL				False	
	TensAOEnable	BOOL				False	
	TensManual_kg	REAL				False	
	TensTaper	REAL				False	
	TensCoreBoost	REAL				False	
	TensRate_kg_s	REAL				False	
	TensMin_kg	REAL				False	
	TensMax_kg	REAL				False	
	TensSet_kg	REAL				False	
	TensAtCore	REAL				False	
	TensAtMid	REAL				False	
	TensAtMax	REAL				False	Metre counter reset button
	TensDiaFrac	REAL				False	
	SonarAskYes	BOOL				False	
	WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped
	FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied onl

						y with motor stopped
SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
RwLearnReset	BOOL				False	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
UnderSpeed	BOOL				False	Speed too low
I_ControlVoltage	BOOL				False	Auxiliaries ON
I_Side_Door_Closed	BOOL				False	Roll diameter photocell
RwCoreDiameter	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SoftStop	BOOL				False	Soft stop in progress (HMI)
SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diame

						ter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Hoover_Mode	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider	BOOL				False	Raise rider roll

_Up						
O_OverRider_Hold	BOOL				False	Lock rider roll
AO_TOne_SetPoint	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_TimeON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_Jog_PB	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
O_RwLoading_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal

Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_at_M_Val	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_Input_11_Spare_1255	BOOL				False	Slow down at end of roll
I_T_one_TensionOK	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
EncoderWheel_Circumf	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement
RwSlowFallbackMm	REAL				False	

AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
RwMatChangePct	REAL				False	
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwStopDiam_Limit	REAL				False	(?) Stop diameter
UwSlow_at_D_Val	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
RwMatReset	BOOL				False	
Psw44	BOOL				False	(?) Password 44
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
UwRawMin	INT				False	
UwRawMax	INT				False	
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwMinPulses	DINT				False	Min pulses in window to learn

RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStop_at_D_Req	BOOL				False	Rewinder diameter STOP request
RwSlow_at_D_Req	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
UwSlow_at_D_Req	BOOL				False	
UwSlowDiamAct	REAL				False	
RwJogMaxDiam	REAL				False	
JogInhibit	BOOL				False	
SCurveTime	REAL				False	
TOneDecel	BOOL				False	
HornStartBeep_s	REAL				False	
HornBeepLen_s	REAL				False	
HornBeepPer	REAL				False	

iod						
HornStopBeep_s	REAL				False	
HornTest	BOOL				False	
RwLockDiam	REAL				False	
UwSensTest	BOOL				False	
RwLockMinGrow_mm	REAL				False	
RwMatLocked	BOOL				False	
LongThick_mm	REAL				False	
RwDiamRev	REAL				False	
RwWave_mm	REAL				False	
UwStopLenShow	BOOL				False	
UwSlowDiamShow	BOOL				False	
SoftCancelAccel	REAL				False	
SpdHold	BOOL				False	
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_M_Val	REAL				False	Unwinder: slow-down starts at (m)
UwSlow_at_M_Req	BOOL				False	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStop_at_D_Val	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	
RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)

RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT				False	Learning: failed stops before alarm
GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
RwThick_mm	REAL				False	Learned paper thickness (mm)
Psw33	BOOL				False	
RwAutoMaxSpd	REAL				False	
RwWinLen_m	REAL				False	
RwWinGrow_mm	REAL				False	
GsmAsk	BOOL				False	
GsmAskYes	BOOL				False	
DecelWeb	REAL				False	
AccelWeb	REAL				False	
RwRatioD	REAL				False	
FFEnable	BOOL				False	
UwStopDiamShow	BOOL				False	
UwStop_at_D_Val	REAL				False	
Clock10ms	BOOL				False	
Sec_to_Min	REAL				False	
Psw48	BOOL				False	
JogRun	BOOL				False	
GuardStopOK	BOOL				False	V11: stopped for real -> guard bypass allowed (Output 13)
SlowDown	BOOL				False	Slow down
WarningOn	bool				False	
AlarmSilenced	BOOL				False	
PaperGSM	REAL				False	
WebWidth_mm	REAL				False	
TensFactor	REAL				False	
TensIdeal_Disp	REAL				False	
TensCheckEnable	BOOL				False	
TensLostWarn	BOOL				False	

	TensWaitSpd	REAL				False	
	TensWait	BOOL				False	
	TensionOK	BOOL				False	
	UwSensFault	BOOL				False	
	Thickness_Fault	BOOL				False	
	AutoStopDone	BOOL				False	
	AutoStopWhy	INT				False	
	AutoEnRefused	BOOL				False	
	AutoEnWhy	INT				False	
	TensWaitHz	REAL				False	
	StartBlocked	BOOL				False	
	GuardOpen	BOOL				False	
	SideDoorOpen	BOOL				False	
	SonarAsk	BOOL				False	
	Psw40	BOOL				False	
	TensJogBoost	REAL				False	
	TensTrim_kg	REAL				False	
	TensJogLatch	BOOL				False	
	CntAutoReset	BOOL				False	
	CntResetMode	INT				False	
	CntResetPend	BOOL				False	
	LastRoll_m	REAL				False	
	RwSlowLearnMin_mm	real				False	

1-8-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



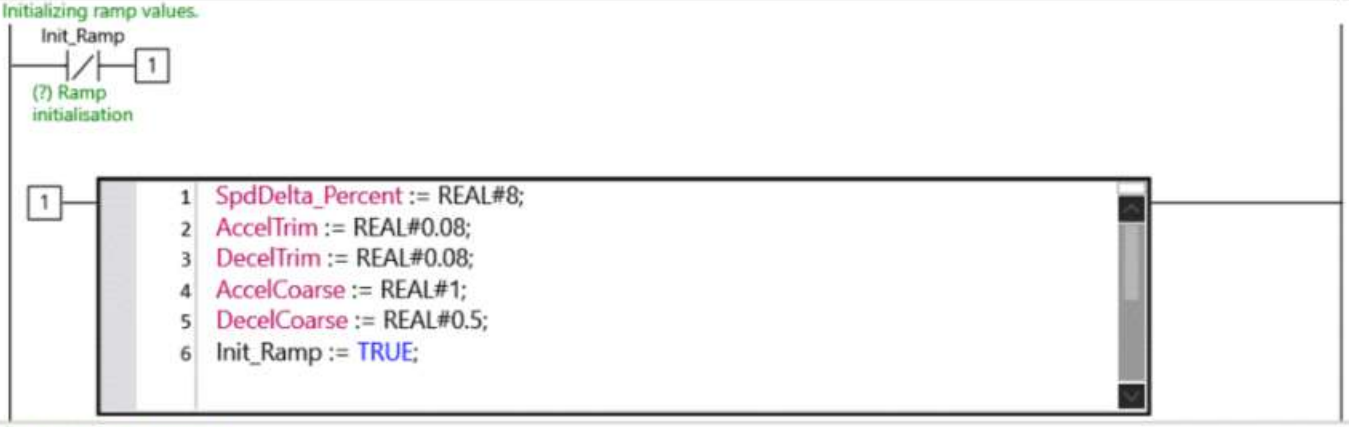
1 100ms clock.



2 10ms clock.



3 Initializing ramp values.



4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.



6 10 V DC source.



7 START blocked by JOG

```
T_JogStartBlk(In := I_Jog_PB, PT := T#2s);      // TOF: ON while held + 2 s
StartHeldBlk := (StartHeldBlk OR (I_Start_PB AND T_JogStartBlk.Q)) AND I_Start_PB;
StartPB_OK := I_Start_PB AND NOT T_JogStartBlk.Q AND NOT StartHeldBlk;

// Side door open (no key bypass): no JOG, and stop any JOG / run in progress

SideDoorOpen := NOT I_Side_Door_Closed AND NOT I_Prot_ByPass;

JogRun := I_Jog_PB AND NOT Slow_Inhibit AND NOT SoftStop AND NOT
SideDoorOpen;

GuardStopOK := NOT StartMotor_Cmd;

// START refused: no paper (monitoring ON), front gate or side door open (no key
bypass)
// JOG: allowed with the front gate open (guard bypass), NOT with the side door
open

WebBreakNow := I_WebBreak_Monitor AND I_WebBreak_PH;
GuardOpen   := (NOT I_FrontGuard_Closed OR NOT I_Side_Door_Closed) AND NOT
I_Prot_ByPass;
StartBlocked := WebBreakNow OR GuardOpen;
```

8

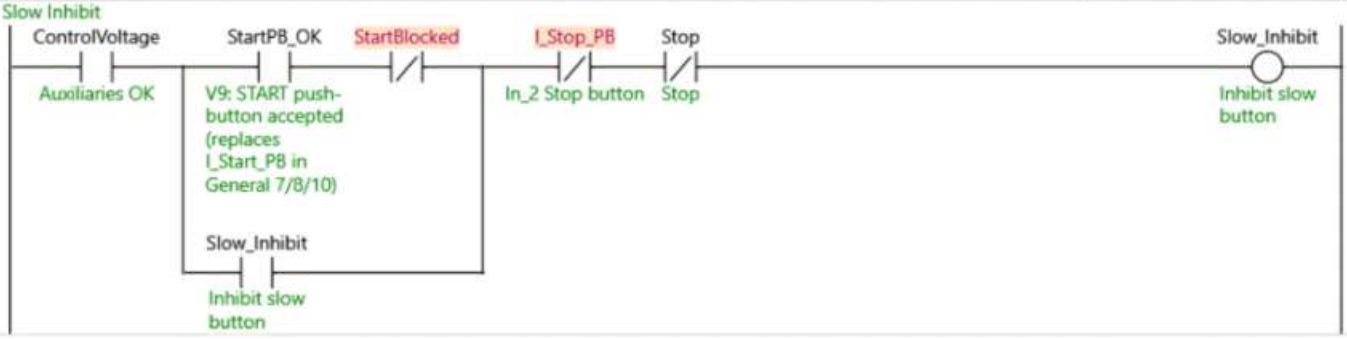
```
T_TensOn(In := I_T_one_TensionOK, PT := T#500ms);      // stable 0.5 s
T_TensOff(In := NOT I_T_one_TensionOK, PT := T#2s);    // ignore short dips

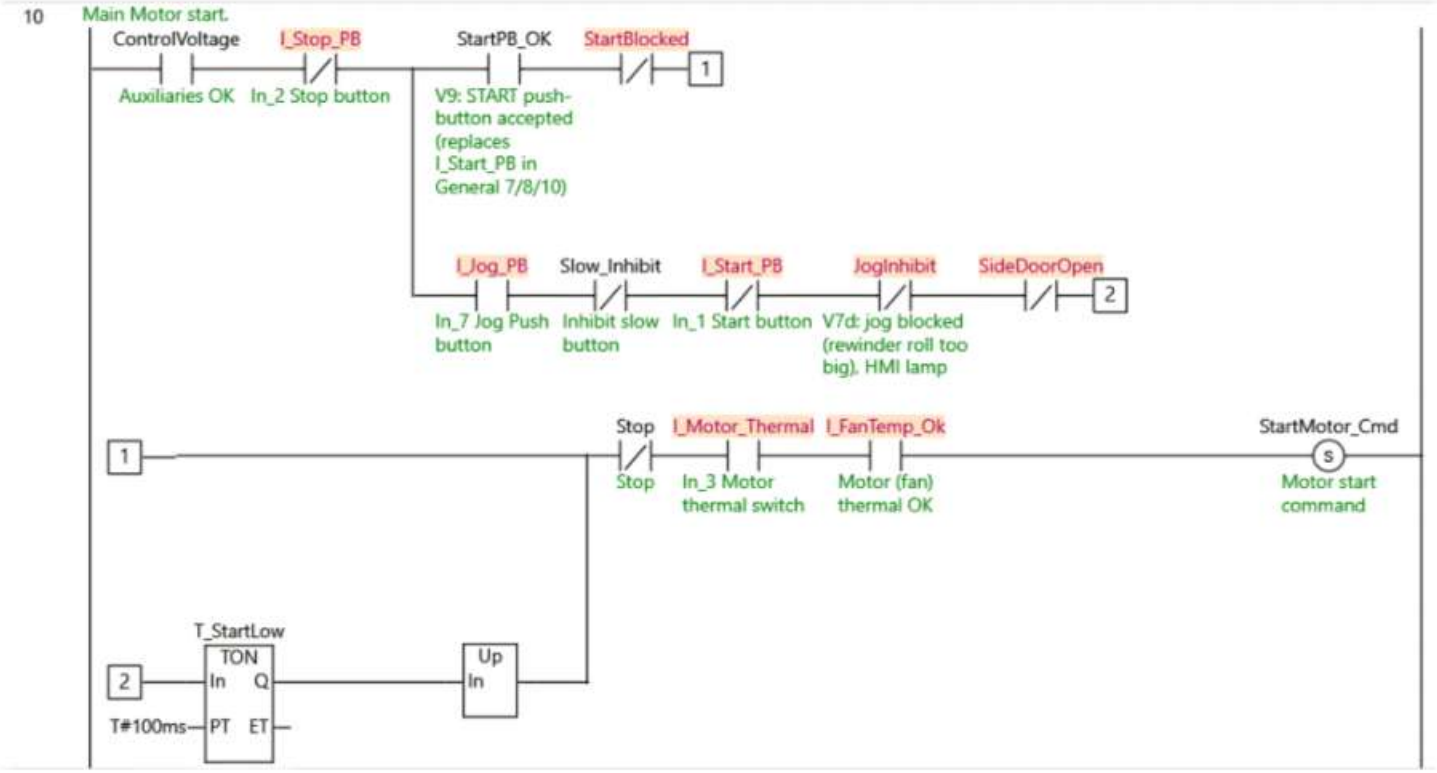
IF T_TensOn.Q OR NOT TensCheckEnable THEN
    TensionOK := TRUE;
ELSIF T_TensOff.Q THEN
    TensionOK := FALSE;
END_IF;

IF NOT StartMotor_Cmd THEN
    TensionSeen := FALSE;
ELSIF TensionOK THEN
    TensionSeen := TRUE;
END_IF;

TensWait := StartMotor_Cmd AND NOT TensionSeen AND NOT JogRun;
TensLostWarn := StartMotor_Cmd AND TensionSeen AND NOT TensionOK;
```

9 Slow Inhibit

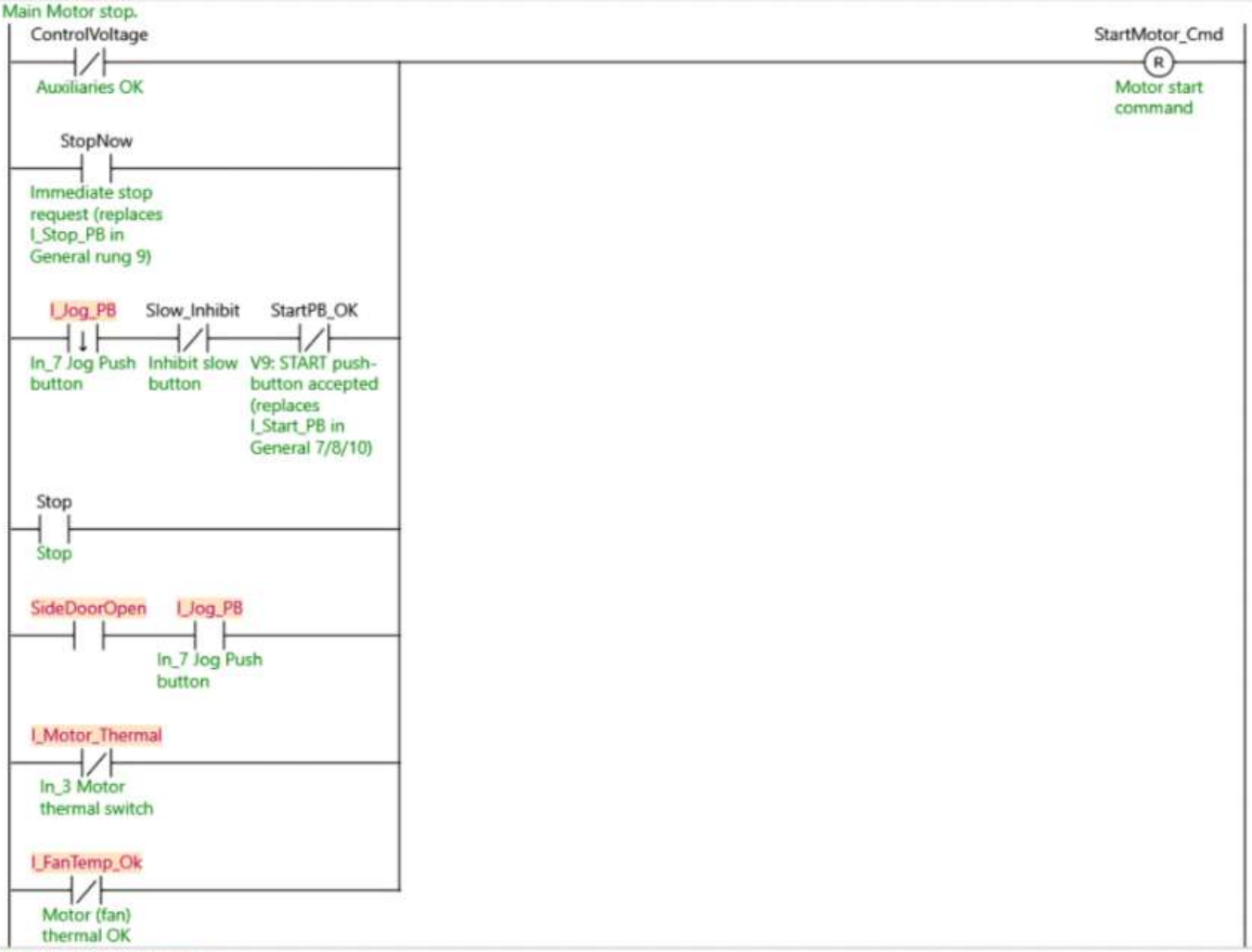



10
Main Motor start.
ControlVoltage
I_Stop_PB
StartPB_OK
StartBlocked
1
Auxiliaries OK
In_2 Stop button
V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
I_Jog_PB
Slow_Inhibit
I_Start_PB
JogInhibit
SideDoorOpen
2
In_7 Jog Push button
Inhibit slow button
In_1 Start button
V7d: jog blocked (rewinder roll too big), HMI lamp
Stop
I_Motor_Thermal
I_FanTemp_Ok
StartMotor_Cmd
1
S
Stop
In_3 Motor thermal switch
Motor (fan) thermal OK
Motor start command
T_StartLow
TON
In
Q
PT
ET
2
T#100ms
Up
In

11 Soft Stop Process

```
1
2  // V12: an end-of-roll stop (FinalStopRamp, AutoStop section 11b) also starts the
3  // soft stop, at FinalStopDecel (Motor_Manager rung 17); START cannot cancel it.
4
5  StopNow := FALSE;
6  T_StopLong(In := I_Stop_PB, PT := T#1s);
7
8  IF NOT StartMotor_Cmd THEN
9     SoftStop := FALSE;
10 ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)
11
12    IF SoftStop OR NOT SoftStopEnable OR I_Jog_PB
13       OR (Velocity_M_min <= SoftStopEndSpd) THEN
14       StopNow := TRUE;                          // 2nd press / jog / slow / disabled
15    ELSE
16       SoftStop := TRUE;                         // start soft stop
17    END_IF;
18 END_IF;
19
20 StopPB_Mem := I_Stop_PB;
21
22 IF T_StopLong.Q THEN
23    StopNow := TRUE;                             // held 1 s = immediate
24 END_IF;
25
26 // V12: end-of-roll stop ramp (FinalStopRamp set by AutoStop section 11b)
27
28 IF FinalStopRamp AND StartMotor_Cmd AND NOT SoftStop THEN
29    SoftStop := TRUE;
30 END_IF;
31
32 // Side door opened while running (not JOG): soft stop, START cannot cancel it
33
34 IF SideDoorOpen AND StartMotor_Cmd AND NOT I_Jog_PB AND NOT SoftStop
   THEN
35    SoftStop := TRUE;
36 END_IF;
37
38 IF SoftStop AND I_Start_PB AND NOT FinalStopRamp AND NOT SideDoorOpen
   THEN
39    SoftStop := FALSE;                           // START cancels the soft stop
40 END_IF;
41
42 T_SoftStop(In := SoftStop, PT := T#120s);
43
44 IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
45    StopNow := TRUE;                             // almost stopped (or timeout)
46 END_IF;
47
```

12 Main Motor stop.



13 Hub extraction clearance.



14 Brake control.

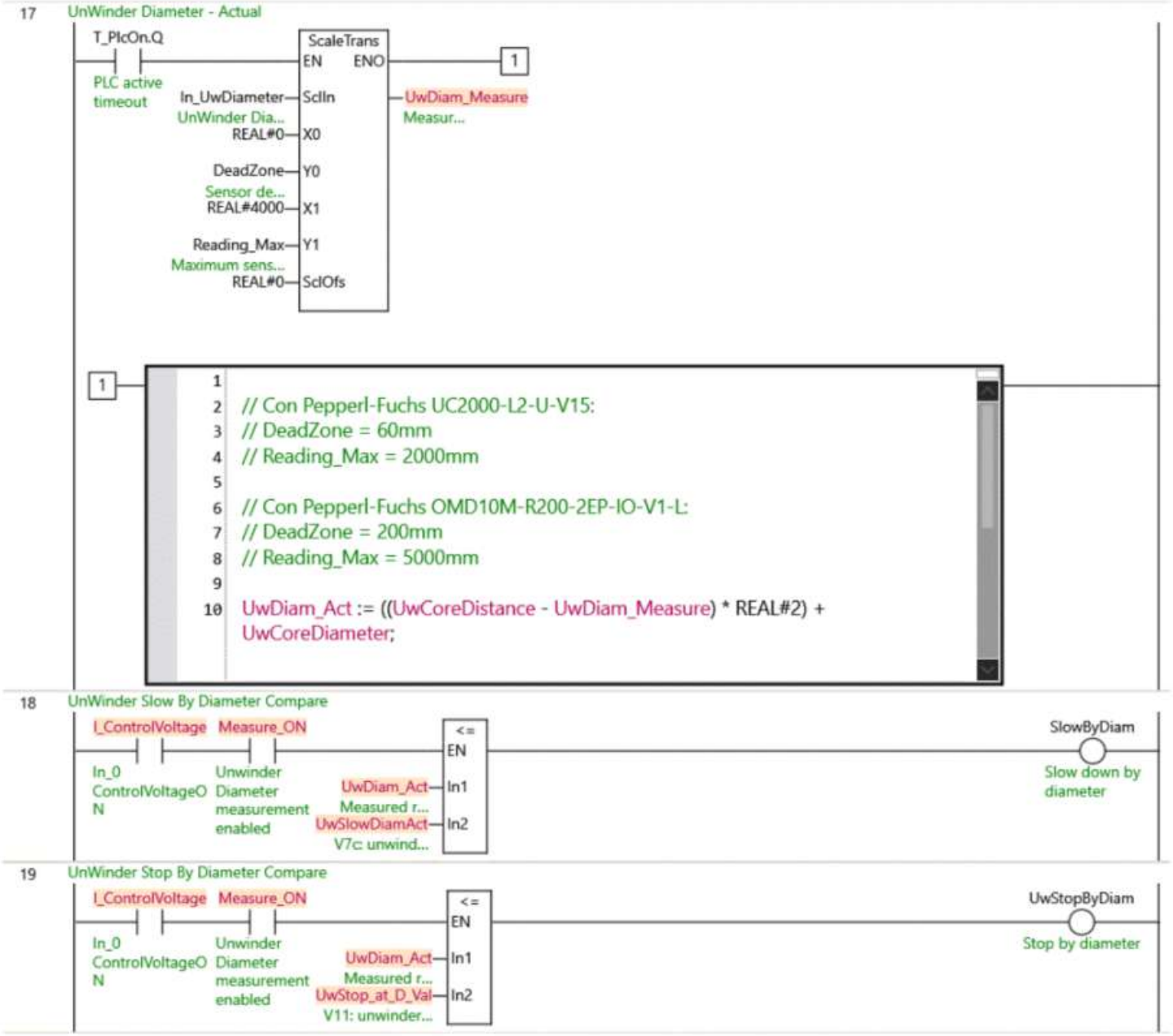


15 Cooling Fan control.



16 UnWinder Diameter - Input

```
1  In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
```


17
UnWinder Diameter - Actual
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
1
In_UwDiameter
UnWinder Dia...
REAL#0
SclIn
X0
UwDiam_Measure
Measur...
DeadZone
Y0
Sensor de...
REAL#4000
X1
Reading_Max
Maximum sens...
REAL#0
Y1
SclOfs
1
// Con Pepperl-Fuchs UC2000-L2-U-V15:
// DeadZone = 60mm
// Reading_Max = 2000mm
// Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
// DeadZone = 200mm
// Reading_Max = 5000mm
UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) + UwCoreDiameter;
18
UnWinder Slow By Diameter Compare
I_ControlVoltage
Measure_ON
In_0 ControlVoltageON
Unwinder Diameter measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDiamAct
V7c: unwind...
In2
SlowByDiam
Slow down by diameter
19
UnWinder Stop By Diameter Compare
I_ControlVoltage
Measure_ON
In_0 ControlVoltageON
Unwinder Diameter measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwStop_at_D_Val
V11: unwinder...
In2
UwStopByDiam
Stop by diameter

1-8-1-1-3.Motor_Manager



1 Calculation of mm per encoder pulse. + Metre counter auto reset at the next START.

```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;

// Encoder Standard 400mm

IF  EncoderWheel_Circumf = REAL#0 THEN;
    EncoderWheel_Circumf := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);

// Rung 1b "Metre counter auto reset at the next START" (ST)  -- PENDING
(Metre_Reset)
// ==== Motor_Manager: NEW ST rung (no contact), directly BEFORE rung 2 ("Reset
encoder counter")
// ==== Comment: "Metre counter auto reset at the next START"
// CntResetMode: 0 = manual only, 1 = with new roll (new parent roll on the
unwinder),
//               2 = after auto-stop.  The event arms CntResetPend; the reset happens
//               with the next accepted START; the old length is kept in LastRoll_m.
// While CntResetPend: the metre AUTO STOP / slow-down / refusal ignore the old
metres.

IF (CntResetMode < 0) OR (CntResetMode > 2) THEN
    CntResetMode := 0;
END_IF;

// unwinder roll was empty (at the stop diameter) since the last reset

IF Measure_ON AND UnWinder_Empty THEN
    CntUwWasEmpty := TRUE;
END_IF;

// new parent roll: stopped, sonar ON, roll no longer empty for 3 s, after an empty
roll

T_CntNewRoll(In := NOT StartMotor_Cmd AND Measure_ON AND CntUwWasEmpty
                AND NOT UnWinder_Empty AND (UwDiam_Act > 0.0), PT := T#3s);

// arm the reset (machine stopped)

IF NOT StartMotor_Cmd
   AND (((CntResetMode = 1) AND T_CntNewRoll.Q) OR ((CntResetMode = 2) AND
AutoStopDone)) THEN
    CntResetPend := TRUE;
    CntUwWasEmpty := FALSE;
END_IF;

IF CntResetMode = 0 THEN
    CntResetPend := FALSE;
END_IF;

// START accepted with a reset pending -> keep the length, 50 ms reset pulse

CntPulseReq := CntResetPend AND StartPB_OK AND NOT CntStartMem AND NOT
StartBlocked:
```

```
51  CntStartMem := StartPB_OK;
52
53  IF CntPulseReq THEN
54      LastRoll_m := Current_Metre;
55      CntPulseDone := TRUE;
56  END_IF;
57
58  T_CntResetPulse(In := CntPulseReq, PT := T#50ms);          // TP
59  CntAutoReset := T_CntResetPulse.Q;
60
61  // pending cleared when the counter is back near 0 (or 2 s after the pulse, safety)
62
63  T_CntDone(In := CntPulseDone AND NOT T_CntResetPulse.Q, PT := T#2s);
64
65  IF CntPulseDone AND NOT T_CntResetPulse.Q
66      AND ((Current_Metre < REAL#1.0) OR T_CntDone.Q) THEN
67      CntResetPend := FALSE;
68      CntPulseDone := FALSE;
69  END_IF;
70
```

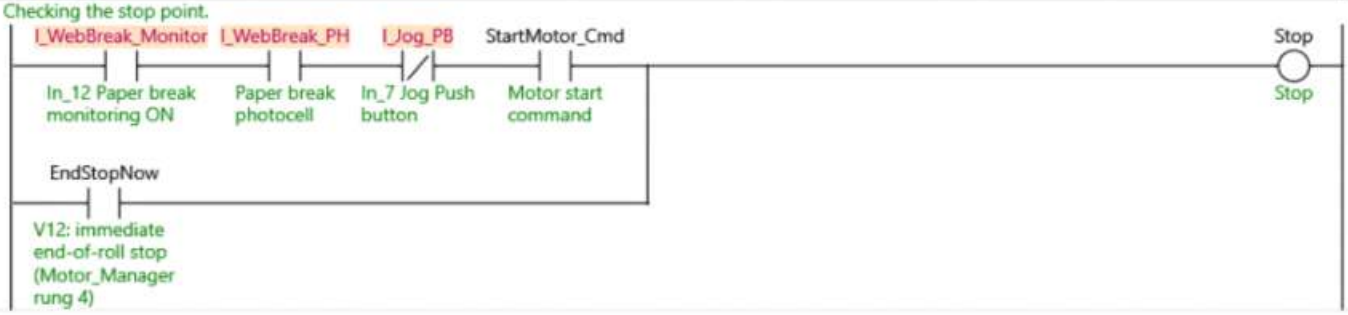
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN
    VelMax := REAL#600;
ELSIF VelMax > REAL#1000 THEN
    VelMax := REAL#1000;
END_IF;

Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN
    JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN
    JogSpd_SP := REAL#70;
END_IF;

// v6y: rewinder AUTO max speed, 0 = no limit

IF RwAutoMaxSpd > VelMax THEN
    RwAutoMaxSpd := VelMax;
ELSIF (RwAutoMaxSpd > REAL#0) AND (RwAutoMaxSpd < REAL#50) THEN
    RwAutoMaxSpd := REAL#50;
END_IF;

IF NOT StartMotor_Cmd THEN       // taken over at standstill only
    RwAutoLimAct := RwStopEnable AND (RwAutoMaxSpd > REAL#0);
    RwAutoMaxSpdAct := RwAutoMaxSpd;
END_IF;

// V7: limits on settings that stop the machine or break the auto stop if 0 / absurd

IF SpdDelta_Percent < REAL#2 THEN
    SpdDelta_Percent := REAL#2;
ELSIF SpdDelta_Percent > REAL#15 THEN
    SpdDelta_Percent := REAL#15;
END_IF;

IF AccelTrim < REAL#0.02 THEN
    AccelTrim := REAL#0.02;
ELSIF AccelTrim > REAL#0.5 THEN
    AccelTrim := REAL#0.5;
END_IF;

IF DecelTrim < REAL#0.02 THEN
    DecelTrim := REAL#0.02;
ELSIF DecelTrim > REAL#0.5 THEN
    DecelTrim := REAL#0.5;
END_IF;

IF AccelCoarse < REAL#0.2 THEN
    AccelCoarse := REAL#0.2;
ELSIF AccelCoarse > REAL#4 THEN
    AccelCoarse := REAL#4;
END_IF;

IF DecelCoarse < REAL#0.2 THEN
    DecelCoarse := REAL#0.2;
ELSIF DecelCoarse > REAL#4 THEN
    DecelCoarse := REAL#4;
```

```
59  END_IF;
60
61  IF DecelWeb > REAL#0 THEN
62
63      IF DecelWeb < REAL#50 THEN
64          DecelWeb := REAL#50;
65      ELSIF DecelWeb > REAL#1000 THEN
66          DecelWeb := REAL#1000;
67      END_IF;
68  END_IF;
69
70  IF AccelWeb > REAL#0 THEN
71
72      IF AccelWeb < REAL#50 THEN
73          AccelWeb := REAL#50;
74      ELSIF AccelWeb > REAL#1000 THEN
75          AccelWeb := REAL#1000;
76      END_IF;
77  END_IF;
78
79  IF SoftStopDecel > REAL#1500 THEN
80      SoftStopDecel := REAL#1500;
81  END_IF;
82
83  IF RwMaxDiam > REAL#1450 THEN
84      RwMaxDiam := REAL#1450;
85  ELSIF RwMaxDiam < REAL#300 THEN
86      RwMaxDiam := REAL#300;
87  END_IF;
88
89  IF RwMaxSlowFallback >= RwMaxDiam THEN
90      RwMaxSlowFallback := RwMaxDiam - REAL#30;
91  END_IF;
92
93  // V7d: jog blocked on a big rewinder roll (0 = no limit, else min 150); DT35 fault ->
    jog allowed
94
95  IF (RwJogMaxDiam > REAL#0) AND (RwJogMaxDiam < REAL#150) THEN
96      RwJogMaxDiam := REAL#150;
97  END_IF;
98
99  JogInhibit := (RwJogMaxDiam > REAL#0) AND (RwDiam > RwJogMaxDiam) AND
    NOT RwSensFault;
100 Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res); //REAL#4000;
101 Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
IF WorkSpd_Max_Set < 50.0 THEN
    WorkSpd_Max_Set := 50.0;
ELSIF WorkSpd_Max_Set > VelMax THEN
    WorkSpd_Max_Set := VelMax;
END_IF;

IF FinalSpd_Max_Set < 5.0 THEN
    FinalSpd_Max_Set := 5.0;
ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
    FinalSpd_Max_Set := WorkSpd_Max_Set;
END_IF;

IF NOT StartMotor_Cmd THEN
    WorkSpd_Max := WorkSpd_Max_Set;          // motor stopped: apply
    FinalSpd_Max := FinalSpd_Max_Set;
END_IF;

SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
            OR (FinalSpd_Max <> FinalSpd_Max_Set)
            OR (RwAutoLimAct <> (RwStopEnable AND (RwAutoMaxSpd > REAL#0)))
            OR (RwAutoLimAct AND (RwAutoMaxSpdAct <> RwAutoMaxSpd));

// v6i: pots scaled to their own full scale (m/min)

IF Pot1_Work_Velocity > INT#5 THEN
    Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max / VelMax);
ELSE
    Pot1_Mem := INT#0;
END_IF;

IF Pot2_Slow_Velocity > INT#5 THEN
    Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max / VelMax);
ELSE
    Pot2_Mem := INT#0;
END_IF;
```

7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;

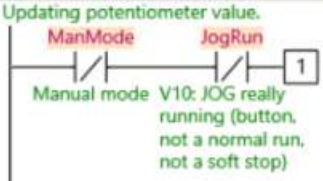
  // v6y: rewinder AUTO stop -> work speed limited

  IF RwAutoLimAct AND (Speed_Display > RwAutoMaxSpdAct) THEN
     Speed_Display := RwAutoMaxSpdAct;
  END_IF;

ELSE;
  Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot, %
  LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);

END_IF;
```

8 Updating potentiometer value.




```
IF SoftStop THEN
   VelSet := 0.0;                    // v6m: soft stop
ELSIF SlowDown = FALSE  THEN
   VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;

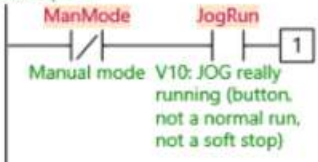
   // v6y: rewinder AUTO stop -> work speed limited

   IF RwAutoLimAct AND (VelSet > RwAutoMaxSpdAct) THEN
      VelSet := RwAutoMaxSpdAct;
   END_IF;

ELSE
   VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;

VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.




```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;

SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;

// Too fast

IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;

// Too slow

IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

11 Speed calculation.

```

// Exactly 25 scans = 100 ms (same as AutoStop SpdAct_Pps), updated 10 times/s.

SpdScanCnt := SpdScanCnt + 1;

IF SpdScanCnt >= 25 THEN
    SpdScanCnt := 0;
    DeltaPulse := Encoder_Pos - MemPos;
    MemPos := Encoder_Pos;

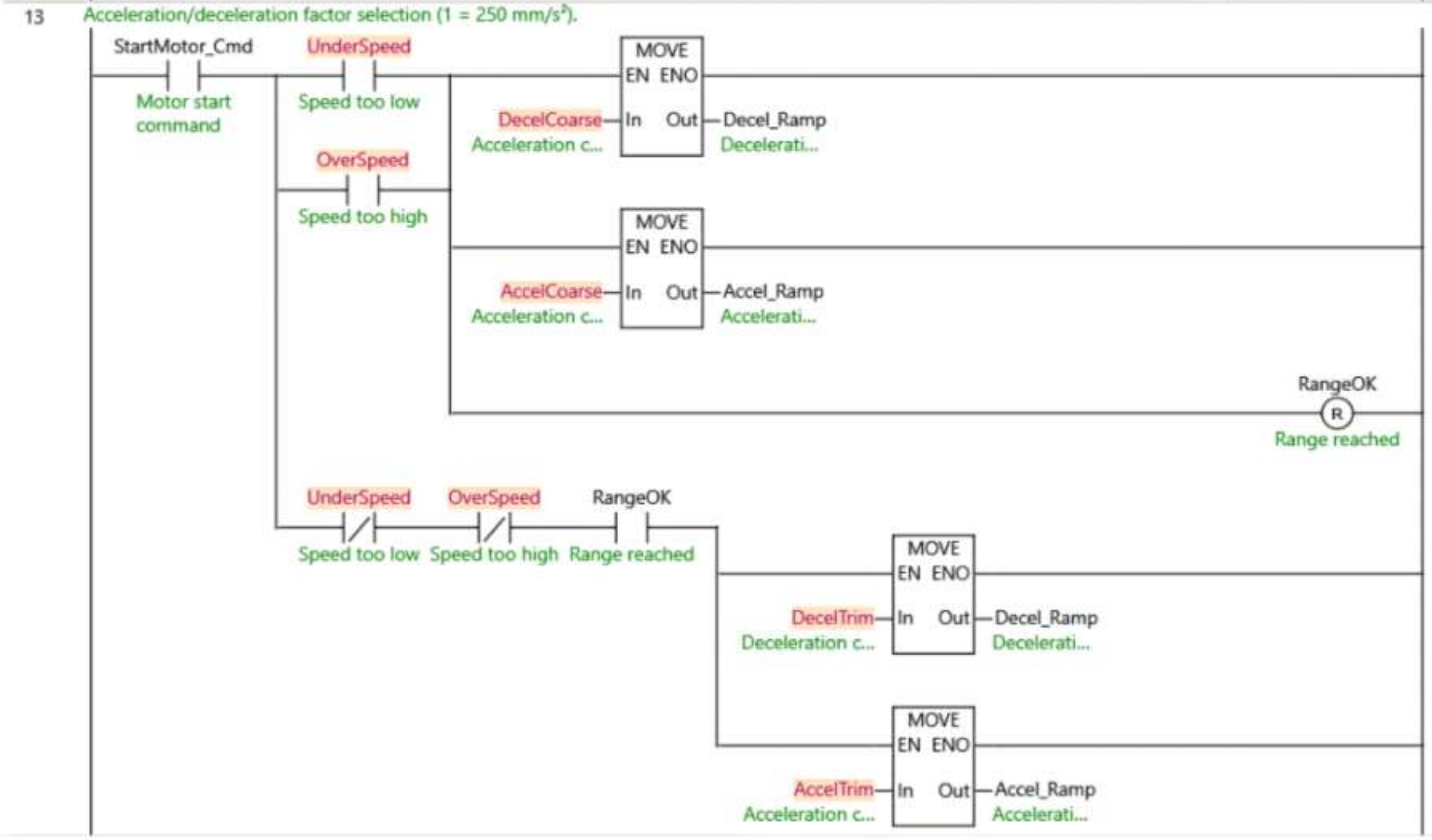
    IF ABS(DeltaPulse) > 100000 THEN            // counter reset (Encoder_Reset)
        DeltaPulse := 0;
    END_IF;

    Velocity := DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm * REAL#10;   // mm/s
    Velocity_M_min := Velocity / Mt_To_mm;
END_IF;
```

12 Set the initial speed.



13 Acceleration/deceleration factor selection (1 = 250 mm/s²).



14 Ratio drive counts per m/min of web at the current rewinder diameter.

```
// ==== V10: REPLACE the code of Motor_Manager rung 14 (printed V9) ("Ratio drive counts per m/min of web
// ==== at the current rewinder diameter." = V7 rung A) with this file. (V7b jog line inside.)
// Ratio drive counts per m/min of web at the current rewinder diameter.
// The motor drives the rewinder: web speed per Hz grows with the diameter, so
// counts per m/min = RwRatioD / diameter. RwRatioD (Retain) is adapted by rung B.

IF (RwRatioD > 0.0) AND (DecelDiam >= 100.0) THEN
    WebRatio := RwRatioD / DecelDiam;
ELSE
    WebRatio := 0.0;
END_IF;

// Fallback only (FFEnable = FALSE): old integrating loop (rung 16) with the coarse
// ramps in real web mm/s2. With FFEnable = TRUE, rung B drives the reference.
// V7b: never in jog -> jog keeps the rung 13 ramps exactly as v6y. V10: JogRun (jog really running).

IF NOT FFEnable AND NOT JogRun AND StartMotor_Cmd AND (UnderSpeed OR OverSpeed)
   AND (WebRatio > 0.0) THEN

   IF DecelWeb > 0.0 THEN
      Decel_Ramp := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;

      IF Decel_Ramp > 8.0 THEN
         Decel_Ramp := 8.0;
      END_IF;
   END_IF;

   IF AccelWeb > 0.0 THEN
      Accel_Ramp := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;

      IF Accel_Ramp > 8.0 THEN
         Accel_Ramp := 8.0;
      END_IF;
   END_IF;
END_IF;
```

15 Soft Stop - SoftStopDecel (mm/s2), same for every roll.

```
// v6r: soft stop at a REAL web decel SoftStopDecel (mm/s2), same for every roll.
// Ratio drive counts / web m/min frozen at the soft stop start (encoder = exact).

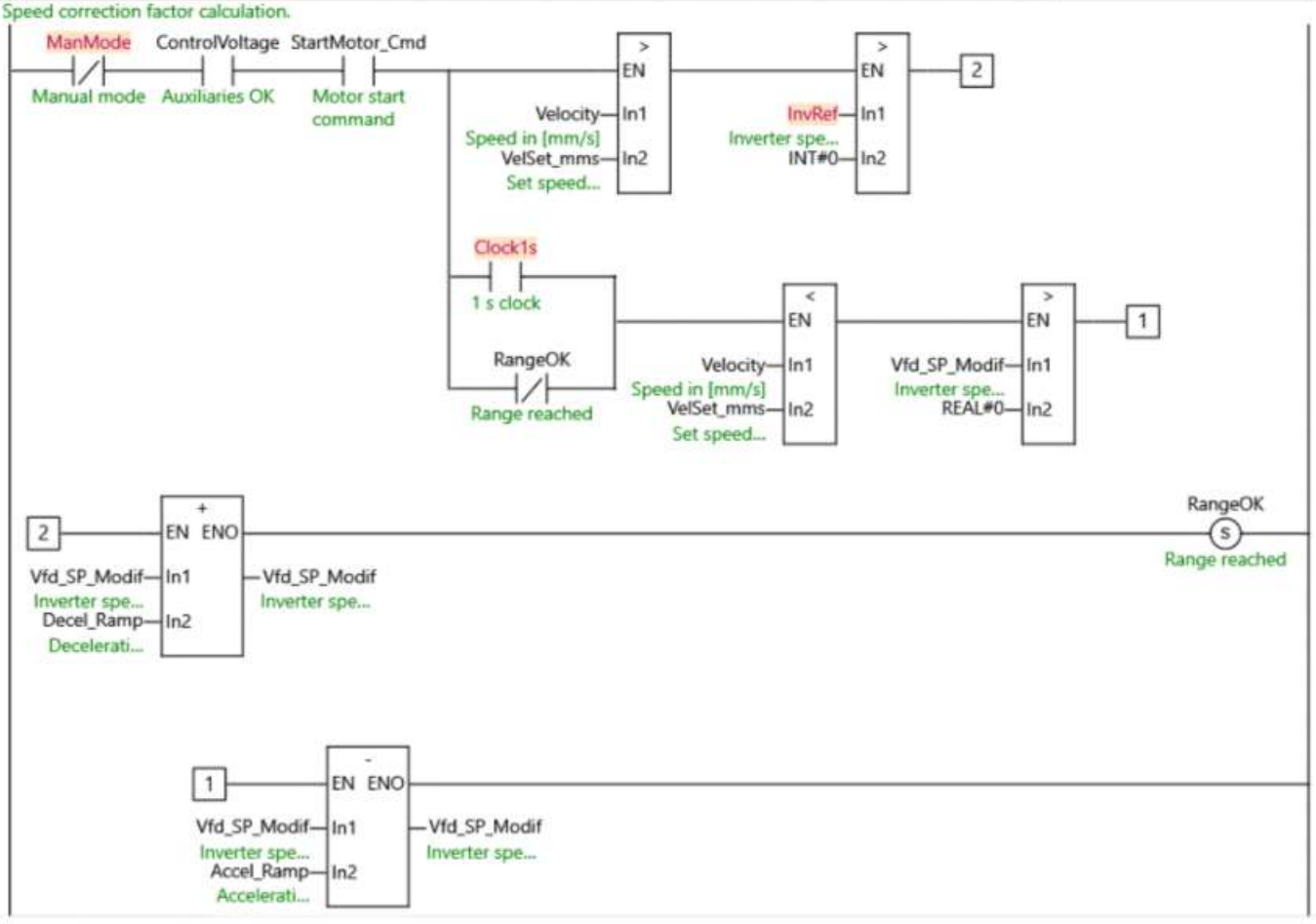
IF SoftStop AND NOT SoftStopMem THEN
    IF Velocity_M_min > 1.0 THEN
        SoftStopRatio := INT_TO_REAL(InvRef) / Velocity_M_min;
    ELSE
        SoftStopRatio := 0.0;
    END_IF;
END_IF;

SoftStopMem := SoftStop;

IF SoftStop AND (SoftStopDecel > 0.0) THEN
    IF SoftStopRatio > 0.0 THEN
        Decel_Ramp := SoftStopDecel * 0.06 * SoftStopRatio * Vel_Max_mms /
1000000.0;
    ELSE
        Decel_Ramp := SoftStopDecel / 250.0;          // fallback (old v6m behaviour)
    END_IF;

    IF Decel_Ramp > 8.0 THEN
        Decel_Ramp := 8.0;                    // max 9 Hz/s asked from the PLC
    END_IF;
END_IF;
```

16 Speed correction factor calculation.



17 Slow adaptation of RwRatioD on the relative speed error.

```
// Rung 17 "Slow adaptation of RwRatioD" = feed-forward (ST, V12 rung B + tension hold)
// ==== V12: REPLACE the code of Motor_Manager rung 17 (printed V9) ("Slow adaptation of RwRatioD on the
// ==== relative speed error." = V7 rung B) with this file. (V7a + V7b + V7f + V9 + V10 + V12 inside.)
// The PLC sends the drive STRAIGHT to the frequency of the set speed
// (VelSet x WebRatio), along a ramp in REAL web mm/s2 that does not depend on the
// measured speed -> no hunting, whatever the roll inertia / web delay.
// The measured speed only adapts RwRatioD slowly (~10 s), once at target.
// It overwrites Vfd_SP_Modif after rung 16, so rung 16 has no effect in AUTO.
// FFEnable = FALSE -> old loop (rung 16) exactly as before.
// V7b: not in jog (V10: JogRun = jog really running): at 10 m/min the feed-forward gives
// only ~0.4 Hz, too little to move the web -> the old loop (rung 16) runs the jog.

FFActive := FFEnable AND NOT ManMode AND NOT JogRun AND StartMotor_Cmd
        AND ControlVoltage AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0);

IF NOT StartMotor_Cmd THEN
    RampRef := 0.0;
    FFRate := 0.0;
    FFSteadyTicks := 0;
    FFSlowAcc := FALSE;
END_IF;

// V9: START during a soft stop: the decel flattens and the speed is HELD
// (SpdHold, HMI lamp) until START is pushed AGAIN -> gentle ramp up at
// SoftCancelAccel (mm/s2) to the set speed. The hold only blocks speeding up:
// a lower target (pot down, auto slow-down, RwAutoMaxSpd) still slows down.
// STOP during the hold = new soft stop (General rung 9).

IF (SoftCancelAccel > 0.0) AND (SoftCancelAccel < 20.0) THEN
    SoftCancelAccel := 20.0;
ELSIF SoftCancelAccel > 1000.0 THEN
    SoftCancelAccel := 1000.0;
END_IF;

FFStartEdge := I_Start_PB AND NOT FFStartMem;
FFStartMem := I_Start_PB;

IF FFSoftMem AND NOT SoftStop AND StartMotor_Cmd AND FFActive THEN   // soft stop cancelled
    SpdHold := TRUE;
    FFSlowAcc := FALSE;
ELSIF SpdHold AND FFStartEdge THEN                    // 2nd START: resume
    SpdHold := FALSE;
    FFSlowAcc := TRUE;
END_IF;

IF SoftStop OR NOT StartMotor_Cmd OR NOT FFActive THEN    // FFEnable OFF: old loop
    SpdHold := FALSE;
    FFSlowAcc := FALSE;
END_IF;

FFSoftMem := SoftStop;
```

```
52
53  // V7f: S-curve time (s), 0 = off, max 2 s
54
55  IF SCurveTime < 0.0 THEN
56      SCurveTime := 0.0;
57  ELSIF SCurveTime > 2.0 THEN
58      SCurveTime := 2.0;
59  END_IF;
60
61  IF FFActive THEN
62
63      IF NOT FFWasActive THEN
64          RampRef := Vel_Max_mms - Vfd_SP_Modif;        // bumpless start / switch-
    over
65          FFRate := 0.0;
66      END_IF;
67
68      // Target reference (mm/s of the 0..10 V scale) for the set speed
69
70      FFTarget := VelSet * WebRatio * Output_Ratio;
71
72      IF FFTarget > Vel_Max_mms THEN
73          FFTarget := Vel_Max_mms;
74      ELSIF FFTarget < 0.0 THEN
75          FFTarget := 0.0;
76      END_IF;
77
78      // Tension: hold at a minimum DRIVE FREQUENCY until the T-one reports tension
79      // (independent of the rewinder diameter). TensWaitHz = 0 -> old m/min hold.
80
81      IF TensWait THEN
82
83          IF TensWaitHz > REAL#0.0 THEN
84              TensRef := TensWaitHz * REAL#4000.0 / REAL#60.0 * Output_Ratio;
85          ELSE
86              TensRef := TensWaitSpd * WebRatio * Output_Ratio;
87          END_IF;
88
89          IF FFTarget > TensRef THEN
90              FFTarget := TensRef;
91          END_IF;
92      END_IF;
93
94      // V9: hold after a cancelled soft stop (the S-curve flattens it smoothly):
95      // never above the present reference, lower targets still allowed
96
97      IF SpdHold THEN
98
99          IF FFTarget > RampRef THEN
100             FFTarget := RampRef;
101         END_IF;
102     ELSIF FFSlowAcc AND (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0) THEN
103         FFSlowAcc := FALSE;                    // back at the set speed
104     END_IF;
105
106     // Ramp steps per 4 ms scan: web mm/s2 -> reference mm/s per scan
107
108     IF FFSlowAcc AND (SoftCancelAccel > 0.0) THEN
109         StepAcc := SoftCancelAccel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;   //
```

```
V9
    ELSIF AccelWeb > 0.0 THEN
        StepAcc := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
    ELSE
        StepAcc := AccelCoarse;
    END_IF;

    IF SoftStop AND FinalStopRamp AND (FinalStopDecel > 0.0) THEN
        StepDec := FinalStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;  //
V12 end-of-roll
    ELSIF SoftStop AND (SoftStopDecel > 0.0) THEN
        StepDec := SoftStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
    ELSIF DecelWeb > 0.0 THEN
        StepDec := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
    ELSE
        StepDec := DecelCoarse;
    END_IF;

    IF StepAcc > 8.0 THEN
        StepAcc := 8.0;
    END_IF;

    IF StepDec > 8.0 THEN
        StepDec := 8.0;
    END_IF;

    // Ramp generator
    // V7f: S-curve. FFRate (reference per scan) changes at most FFJerk per scan,
    // so the full ramp is reached after SCurveTime (s) and is rounded off the
    // same way before the target -> the T-one / brake air can follow.
    // SCurveTime = 0 -> plain linear ramp as V7a.

    IF SCurveTime > 0.0 THEN

        IF (FFRate > 0.0) OR ((FFRate = 0.0) AND (FFTarget > RampRef)) THEN
            FFJerk := StepAcc * 0.004 / SCurveTime;
        ELSE
            FFJerk := StepDec * 0.004 / SCurveTime;
        END_IF;

        // rate allowed so that it can still fall to 0 at the target
        // (no MIN(), argument in a REAL variable first: Sysmac data type rules,
        //  same pattern as SqrtArg / SQRT(SqrtArg) in AutoStop)

        IF FFTarget >= RampRef THEN
            FFSqrtArg := REAL#2.0 * FFJerk * (FFTarget - RampRef);
            FFRateSet := SQRT(FFSqrtArg);

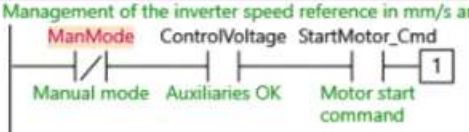
            IF FFRateSet > StepAcc THEN
                FFRateSet := StepAcc;
            END_IF;
        ELSE
            FFSqrtArg := REAL#2.0 * FFJerk * (RampRef - FFTarget);
            FFRateSet := SQRT(FFSqrtArg);

            IF FFRateSet > StepDec THEN
                FFRateSet := StepDec;
            END_IF;
```

```
167         END_IF;
168
169         IF FFRateSet > FFRate + FFJerk THEN
170             FFRate := FFRate + FFJerk;
171         ELSIF FFRateSet < FFRate - FFJerk THEN
172             FFRate := FFRate - FFJerk;
173         ELSE
174             FFRate := FFRateSet;
175         END_IF;
176
177         RampRef := RampRef + FFRate;
178
179         IF (ABS(FFTarget - RampRef) <= 3.0) AND (ABS(FFRate) <= 2.0 * FFJerk) THEN
180             RampRef := FFTarget;                    // at target (pot noise)
181             FFRate := 0.0;
182         END_IF;
183     ELSE
184         FFRate := 0.0;
185
186         IF RampRef < FFTarget - StepAcc THEN
187             RampRef := RampRef + StepAcc;
188         ELSIF RampRef > FFTarget + StepDec THEN
189             RampRef := RampRef - StepDec;
190         ELSE
191             RampRef := FFTarget;                    // at target
192         END_IF;
193     END_IF;
194
195     // V7a: "at target" = within 0.5 % of the target (the pot and RwDiam noise
196     // move FFTarget by a few counts every scan -> the counter never reached 500)
197
198     IF (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0)
199       AND NOT SpdHold AND NOT FFSlowAcc AND NOT TensWait THEN          // V9:
    not during the hold
200
201         IF FFSteadyTicks < 30000 THEN
202             FFSteadyTicks := FFSteadyTicks + 1;
203         END_IF;
204     ELSE
205         FFSteadyTicks := 0;
206     END_IF;
207
208     // Slow adaptation of RwRatioD on the relative speed error (2 s after the
209     // target is reached, not at the 60 Hz limit, not during soft stop / jog)
210
211     IF (FFSteadyTicks >= 500) AND NOT SoftStop AND NOT JogRun
212       AND (VelSet > 15.0) AND (Velocity_M_min > 10.0) AND NOT RwSensFault
213       AND (FFTarget < Vel_Max_mms * 0.98) THEN
214         FFErr := (VelSet - Velocity_M_min) / VelSet;
215
216         IF FFErr > 0.05 THEN
217             FFErr := 0.05;
218         ELSIF FFErr < -0.05 THEN
219             FFErr := -0.05;
220         END_IF;
221
222         IF ABS(FFErr) > 0.003 THEN                  // dead band 0.3 % (encoder noise)
223             RwRatioD := RwRatioD * (1.0 + 0.0004 * FFErr);  // ~10 s time constant
```

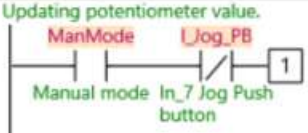
```
224     END_IF;
225
226     IF RwRatioD < 1000.0 THEN
227       RwRatioD := 1000.0;
228     ELSIF RwRatioD > 8000.0 THEN
229       RwRatioD := 8000.0;
230     END_IF;
231   END_IF;
232
233   Vfd_SP_Modif := Vel_Max_mms - RampRef;          // rung 17 -> InvRef
234 END_IF;
235
236 FFWasActive := FFActive;
237
238 // V7f: T-one decel signal (for a T-one "stop / boost" input, spare PLC output):
239 // ON while the feed-forward ramps down (more than 1 % above the target, so the
240 // slow drift of the target with the diameter does not pulse it) or soft stop.
241
242 TOneDecel := (FFActive AND (RampRef - FFTarget > FFTarget * 0.01 + 5.0))
243         OR (SoftStop AND StartMotor_Cmd);
```

18 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10.000 mm/s).

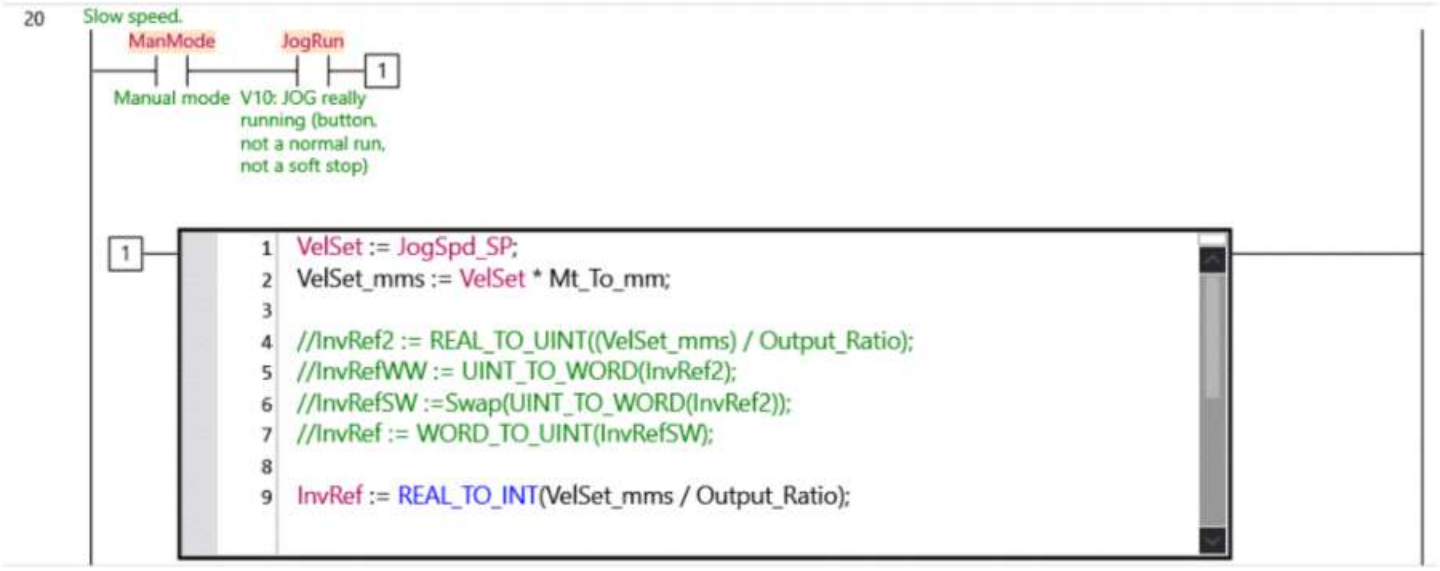



```
1  //InvRef2 := REAL_TO_UINT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
2  //InvRefWW := UINT_TO_WORD(InvRef2);
3  //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
4  //InvRef := WORD_TO_UINT(InvRefSW);
5
6  InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

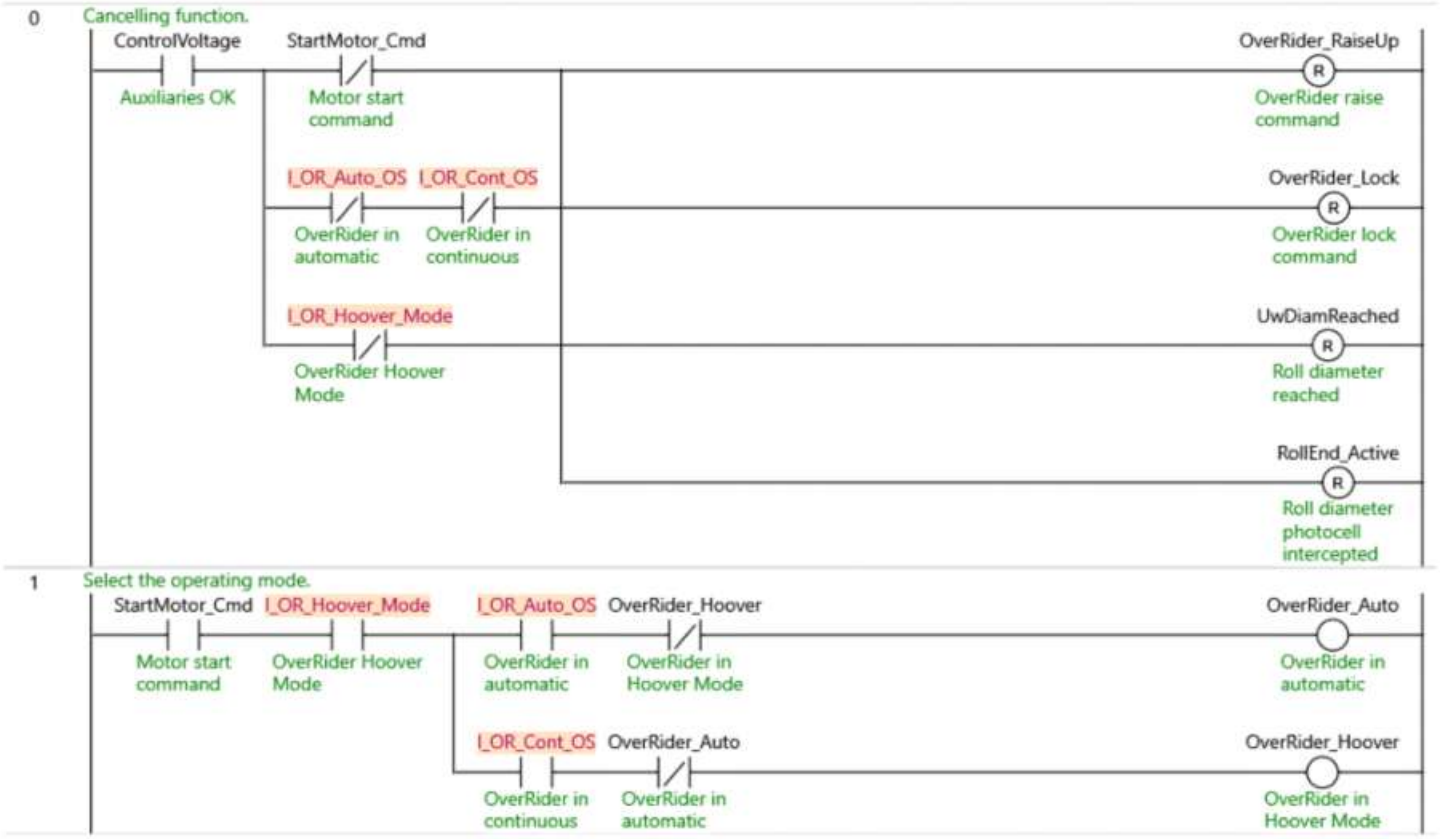
19 Updating potentiometer value.



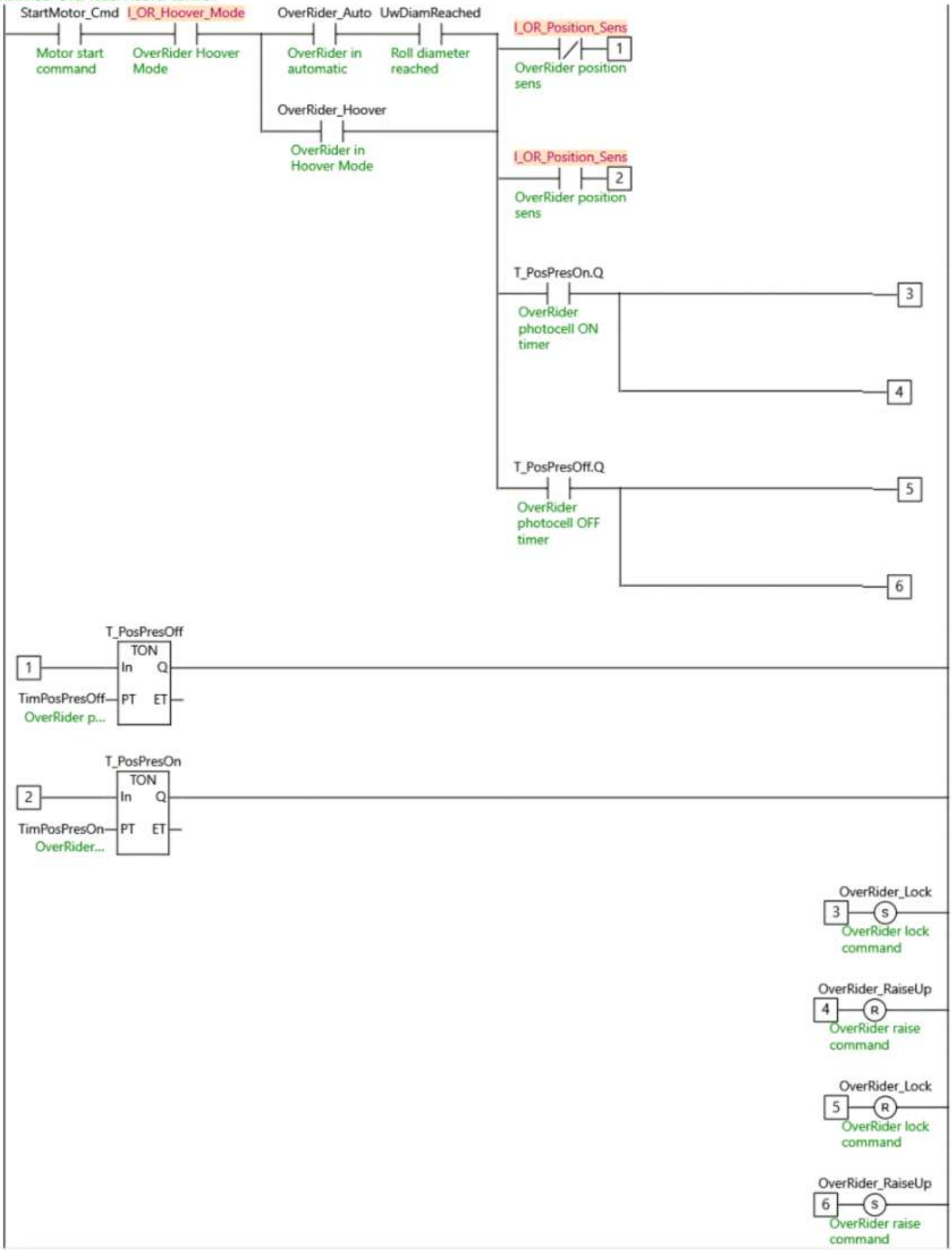

```
1  IF SoftStop THEN
2    InvRef := INT#0;                    // v6m: drive ramps down on its own
3  ELSIF SlowDown = FALSE THEN
4    InvRef := Pot1_Work_Velocity;
5  ELSE
6    InvRef := Pot2_Slow_Velocity;
7  END_IF;
```


20
Slow speed.
ManMode
JogRun
1
Manual mode
V10: JOG really running (button, not a normal run, not a soft stop)
1
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3
4 //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
5 //InvRefWW := UINT_TO_WORD(InvRef2);
6 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
7 //InvRef := WORD_TO_UINT(InvRefSW);
8
9 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);

1-8-1-1-4.OverRider_Manager



2 Rewinder Over Rider Hoover control.



3 Rider roll raise command.



4



1-8-1-1-5.Console

0 Variables of type 'TIME'.

```
1  TimStopVen := NanoSecToTime(REAL_TO_LINT(Fan_Timeout * REAL#60000000000));
2  TimPosPresOn := NanoSecToTime(REAL_TO_LINT(OverRider_TimeON *
   REAL#1000000000));
3  TimPosPresOff := NanoSecToTime(REAL_TO_LINT(OverRider_TimeOFF *
   REAL#1000000000));
```

1 Convert the pulses into mm and m.

```
1  Current_mm := DINT_TO_REAL(Encoder_Pos) * EncoderPulse_to_mm;
2  Current_Metre := Current_mm / REAL#1000;
```

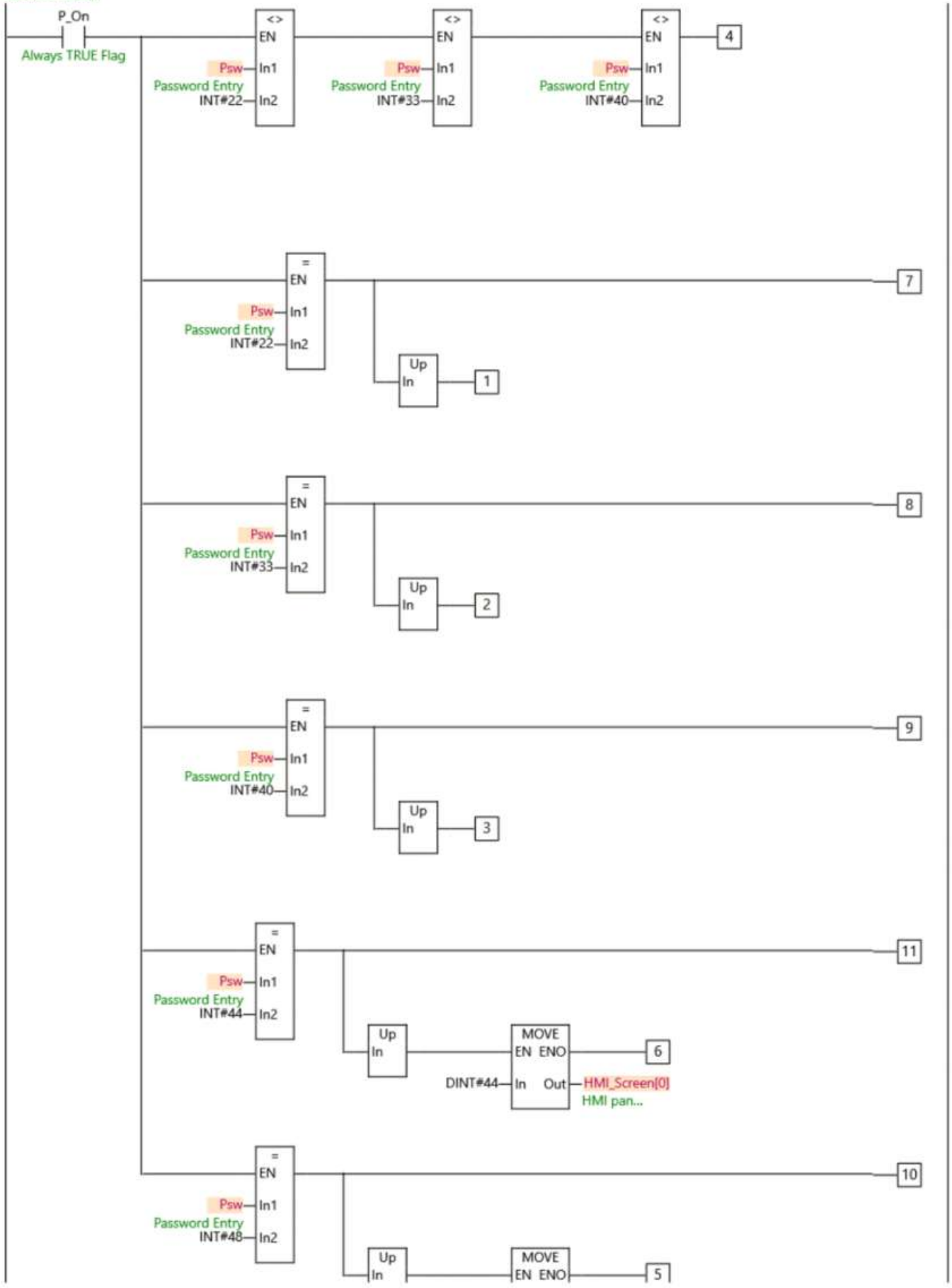
2 Motor Stop

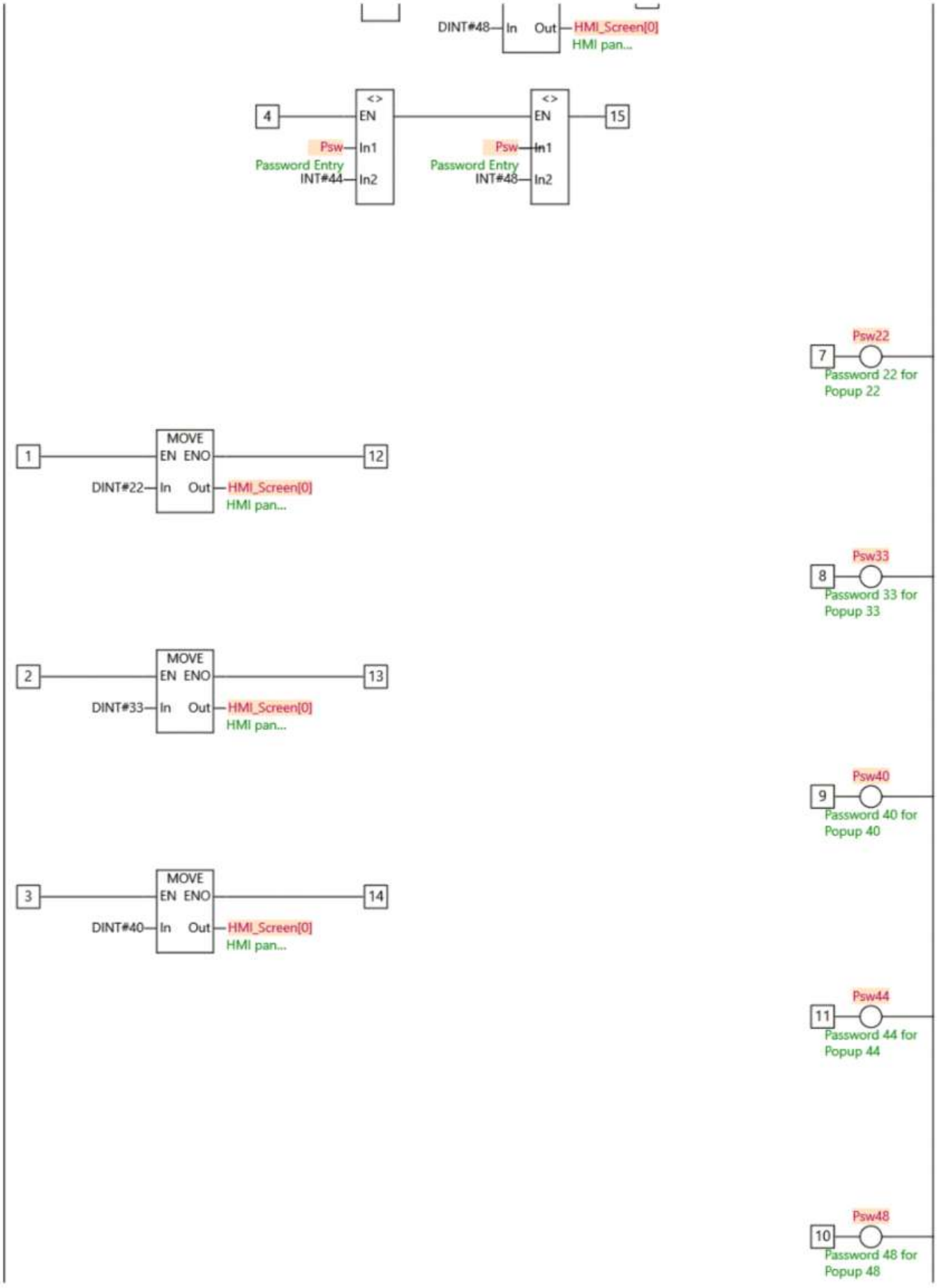


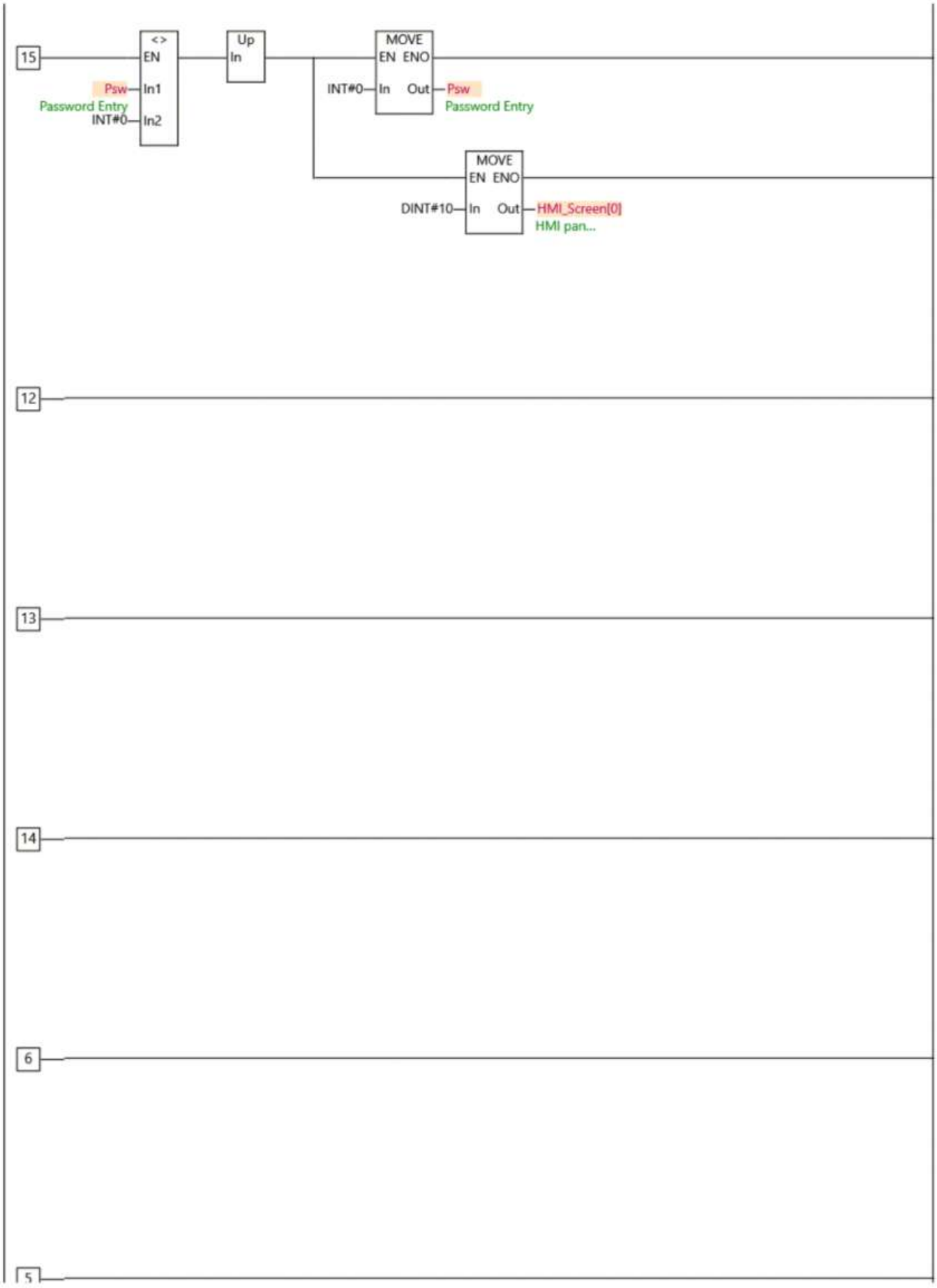
3 Request to reset the meter counter.

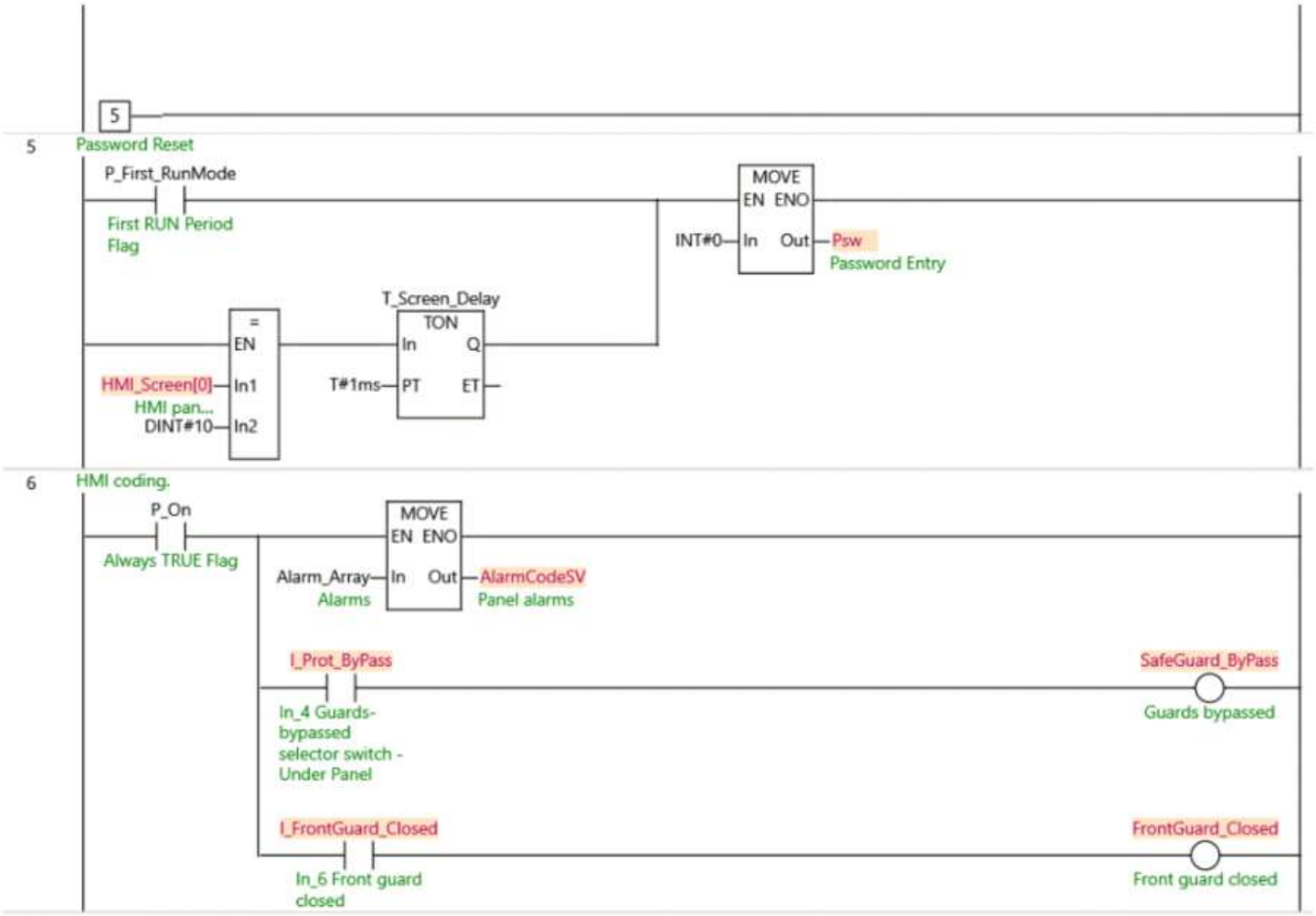


4 Password check..




DINT#48
In
Out
HMI_Screen[0]
HMI pan...
4
<>
EN
Psw
Password Entry
INT#44
In1
In2
15
INT#48
Psw22
7
Password 22 for Popup 22
MOVE
EN ENO
1
12
DINT#22
Psw33
8
Password 33 for Popup 33
2
13
DINT#33
Psw40
9
Password 40 for Popup 40
3
14
DINT#40
Psw44
11
Password 44 for Popup 44
Psw48
10
Password 48 for Popup 48


15
<>
EN
Psw
Password Entry
In1
INT#0
In2
Up
In
MOVE
EN ENO
INT#0
In
Out
Psw
Password Entry
MOVE
EN ENO
DINT#10
In
Out
HMI_Screen[0]
HMI pan...
12
13
14
6
5


5
5
Password Reset
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In
Out
Psw
Password Entry
T_Screen_Delay
=
EN
In1
In2
HMI_Screen[0]
HMI pan...
DINT#10
TON
In
Q
T#1ms
PT
ET
6
HMI coding.
P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarms
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
In_4 Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
In_6 Front guard closed
FrontGuard_Closed
Front guard closed

1-8-1-1-6.Alarms

0 Check Present alarms.

```
// Alarm / warning handling + panel buzzer (BuzzerOn).
// Alarms: buzzer pulses (Clock1s). ALARM RESET (panel or HMI) silences it 1 min;
//         a NEW alarm during the silence sounds again at once.
// Warnings: ONE 1 s beep each time a new warning appears.
// PrimaryTask 4 ms: 250 scans = 1 s, 15000 scans = 60 s.

AlarmOn := FALSE;
WarningOn := FALSE;
AlarmCnt := 0;
WarnCnt := 0;

FOR Index_A := 0 TO 32 DO

   IF Alarm_Array[Index_A] THEN

      CASE Index_A OF
         5, 13, 15, 16, 17, 18, 19, 20, 21:        // warnings
            WarningOn := TRUE;
            WarnCnt := WarnCnt + 1;

            IF NOT AlarmMem[Index_A] THEN
               WarnBeepTicks := 250;              // NEW warning -> 1 s beep
            END_IF;
      ELSE                                        // alarms
         AlarmOn := TRUE;
         AlarmCnt := AlarmCnt + 1;
      END_CASE;

   END_IF;
   AlarmMem[Index_A] := Alarm_Array[Index_A];
END_FOR;

// ALARM RESET button (panel I_RsetAlarm or HMI Alarm_Reset): silence 1 min

RstEdge := (I_RsetAlarm OR Alarm_Reset) AND NOT RstMem;
RstMem := I_RsetAlarm OR Alarm_Reset;

IF RstEdge AND AlarmOn THEN
   SilenceTicks := 15000;
END_IF;

IF (AlarmCnt > AlarmCntMem) OR NOT AlarmOn THEN
   SilenceTicks := 0;                             // new alarm / no alarm: not silenced
END_IF;

IF SilenceTicks > 0 THEN
   SilenceTicks := SilenceTicks - 1;
END_IF;

IF WarnBeepTicks > 0 THEN
   WarnBeepTicks := WarnBeepTicks - 1;
END_IF;

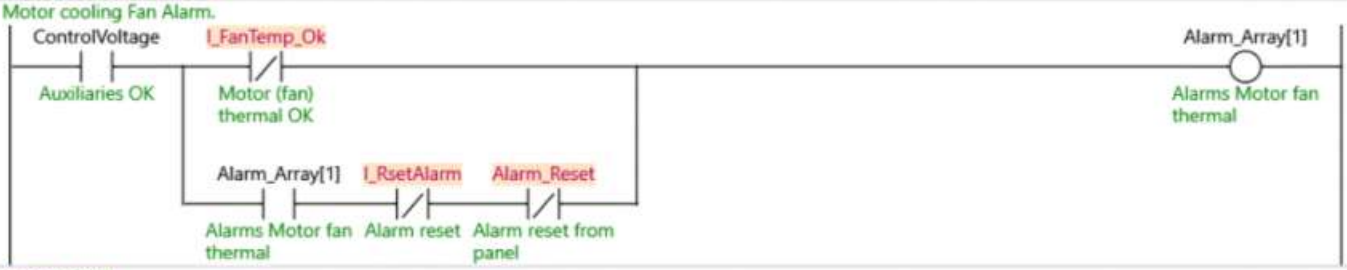
AlarmCntMem := AlarmCnt;
WarnCntMem := WarnCnt;
```

```
57  AlarmSilenced := AlarmOn AND (SilenceTicks > 0);
58
59  BuzzerOn := ControlVoltage
60          AND ((AlarmOn AND NOT AlarmSilenced AND Clock1s) OR (WarnBeepTicks
    > 0));
```

1

```
1   // ---- Alarm 14 (ALARM): unwinder sonar (NX-AD2603 Ch1, P+F UC2000) fault ----
2   // Raw value out of range for 1 s (no echo / broken wire / sensor off).
3   // Set when the measurement is used (unwinder AutoStop ON or motor running),
4   // latched, RESET only once the sensor is OK again (same as alarm 10, DT35).
5
6   T_UwSensFlt(In := (AI_Diam_Unwinder < UwRawMin) OR (AI_Diam_Unwinder >
    UwRawMax), PT := T#1s);
7   UwSensTest := T_UwSensFlt.Q;
8
9   IF ControlVoltage AND Measure_ON AND UwSensTest AND (UwStopEnable OR
    StartMotor_Cmd) THEN
10     UwSensFault := TRUE;
11  ELSIF NOT UwSensTest AND (I_RsetAlarm OR Alarm_Reset) THEN
12     UwSensFault := FALSE;
13  END_IF;
14
15  // ---- Alarm 17 (WARNING): Paper thickness not learned ----
16  // RwMat_K = 0 -> diameter / stop-length targets use the fixed fallback.
17  // Only above 300 mm rewinder diameter (learning needs ~20 mm growth after
    NEW PAPER),
18  // so it does not beep at every new roll. Not latched.
19
20  Thickness_Fault := ControlVoltage AND StartMotor_Cmd
21              AND (RwMat_K <= 0.0) AND (RwDiam > 300.0);
```

2 Motor cooling Fan Alarm.



3 Inverter fault.



4 Machine in emergency mode.



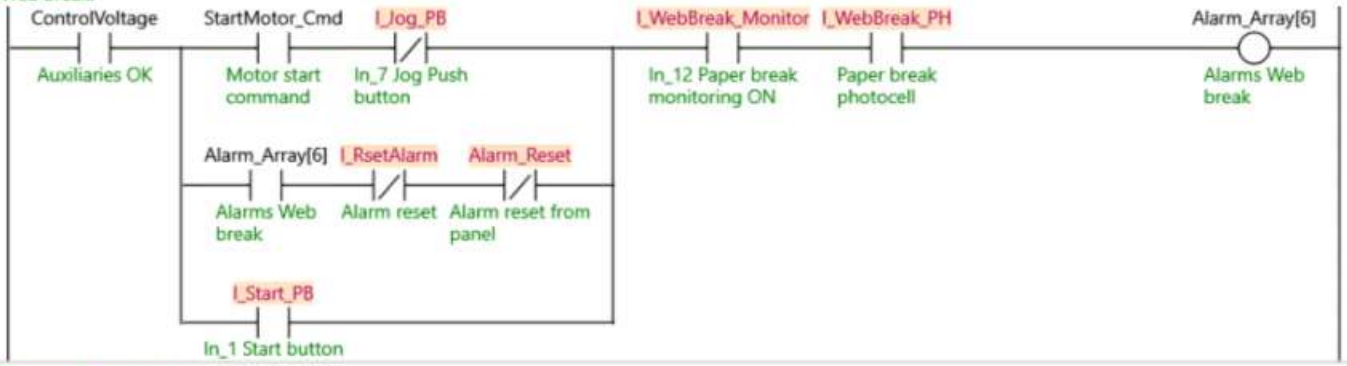
5 Main Motor thermal cutout.



6 Front Guard - Warning



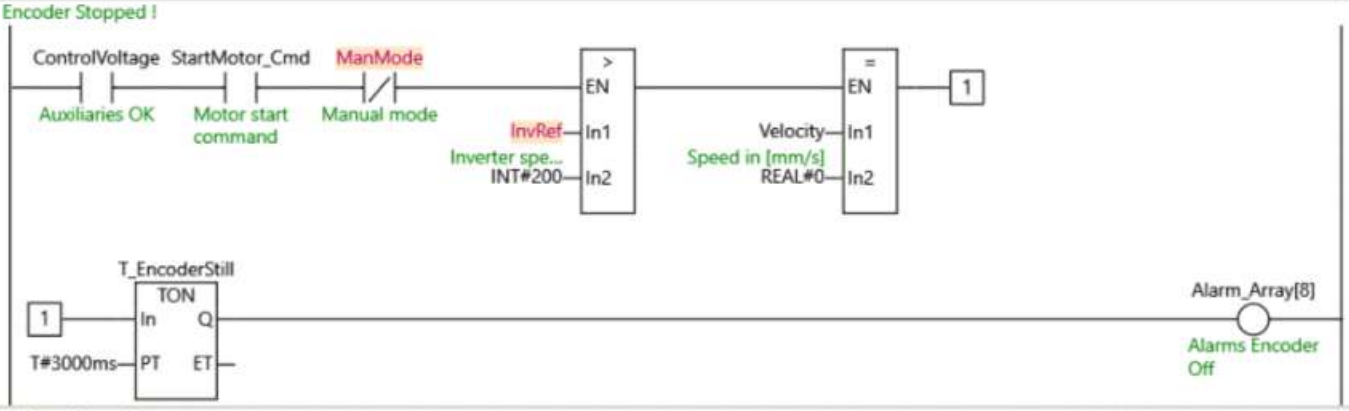
7 Web Break.



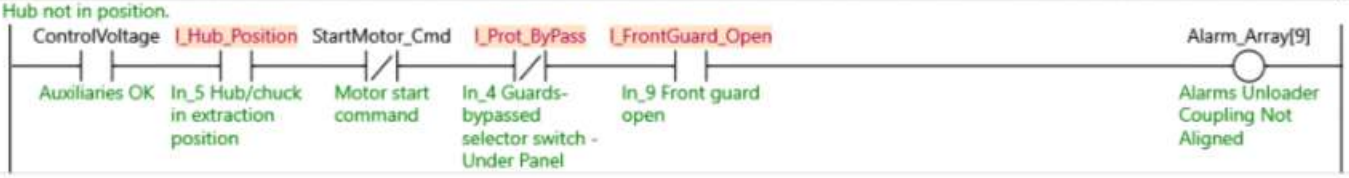
8 Vacuum Cooling fan.



9 Encoder Stopped !



10 Hub not in position.



11 Rewinder Diameter Sensor Fault.



12 Rewinder Diameter Sensor Fault - Reset



13 Rewinder max diameter Machine protection.



14 Rewinder max diameter Machine protection - Reset



15 Emergency Stop by Holding Stop PB 2s



16 Web Tension Monitor - Warning



17 Unwinder Diameter Sensor Fault.



18 Unwinder Roll is Empty - Warning



19 Unwinder Roll is Empty - Reset



20 Rewinder GSM learning fault



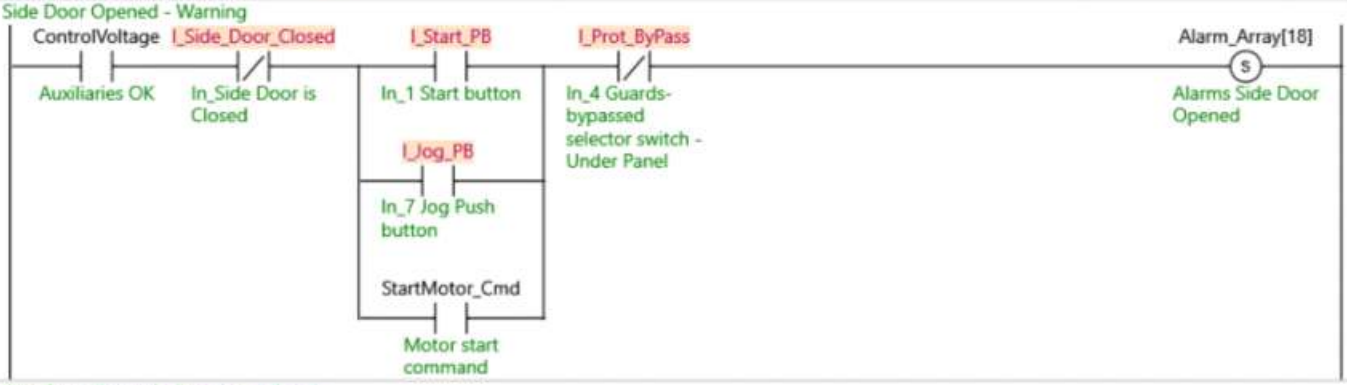
21 Rewinder learning fault - Reset



22 Paper Thickness Learn Process - Not Done ! - Warning



23 Side Door Opened - Warning



24 Side Door Opened - Warning - Reset



25 Front Gate Not Closed ! - Warning



26 Front Gate Not Closed ! - Warning - Reset



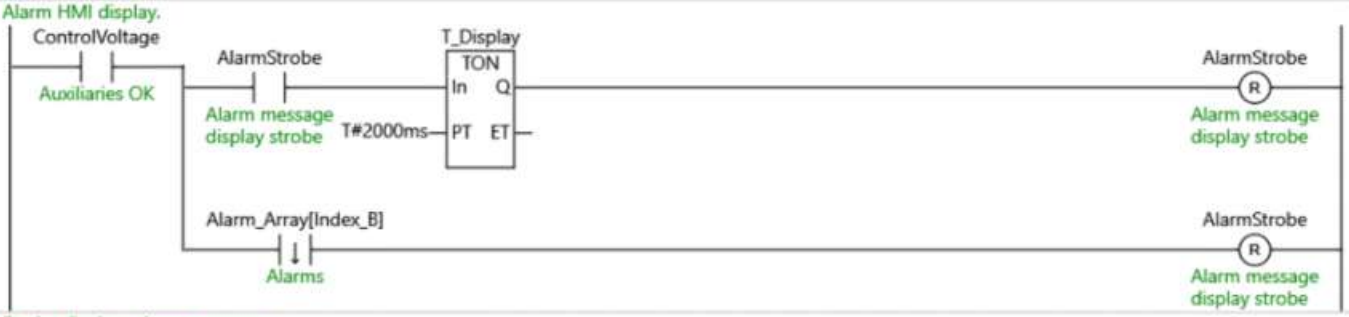
27 AutoStop Done ! - Warning



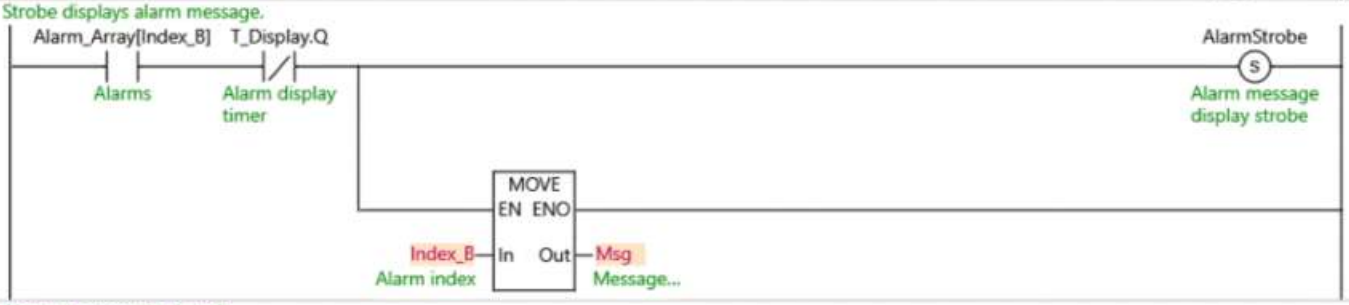
28 AutoStop Set Enable Monitor - Warning



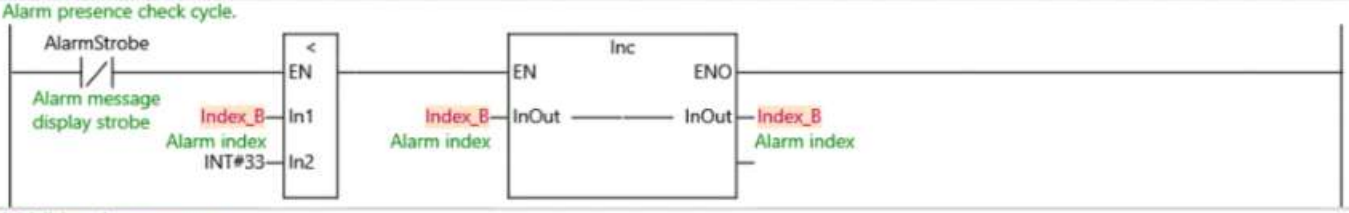
29 Alarm HMI display.



30 Strobe displays alarm message.



31 Alarm presence check cycle.

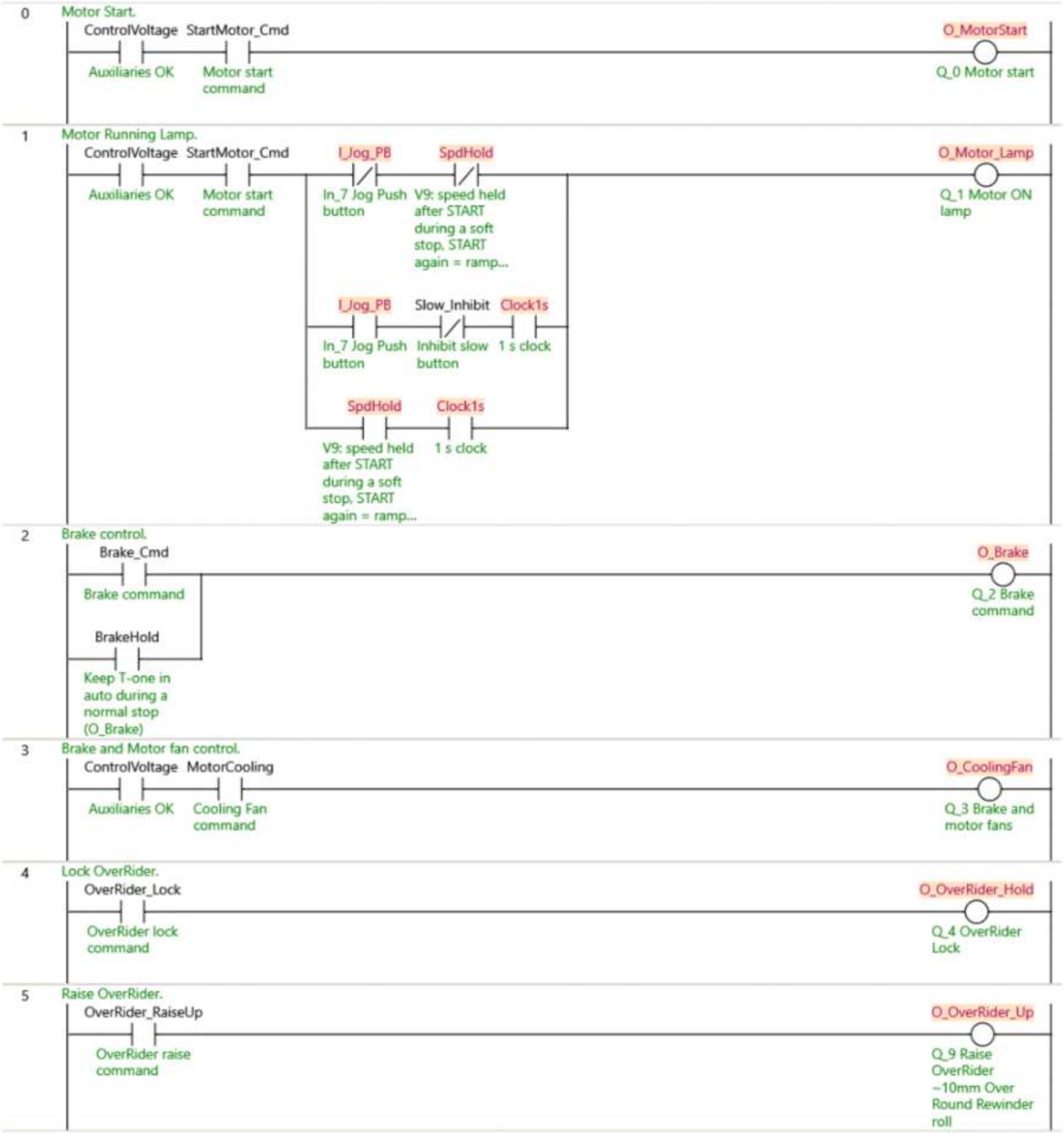


32 Initializing alarm message.




33
AlarmOn
Alarm active
WarningOn
HMI Warning Display
MOVE
EN ENO
INT#33
In
Out
Msg
Message...

1-8-1-1-7.Output_Manager



6 Rider roll auto/continuous lamp.



7 Alarm lamp.



8 Guards bypassed lamp.



9 Unloaders enable - opening.



10 Unloaders enable - closing.



11 Horn / acoustic signal.

```
// ==== V9: REPLACE Output_Manager rung 11 ("Horn / acoustic signal.", ladder) by ONE ST rung
// ==== with this code (no contact in front). Comment: "V9: horn, beeps set on the HMI".
// Old: horn blinking with Clock1s (0.5 s ON / 0.5 s OFF) during the whole slow-down
//      and the crawl at final speed -> very annoying.
// New, slow-down / final speed (StartMotor_Cmd AND SlowDown AND NOT Stop):
//      HornStartBeep_s beep when it starts (1 s),
//      then a HornBeepLen_s beep (0.2 s) every HornBeepPeriod (5 s),
//      then ONE HornStopBeep_s beep (3 s) when the machine has stopped (< 2 m/min).
//      Each setting 0 = that beep OFF. Front guard moving: unchanged (Clock1s blinking).
// PrimaryTask 4 ms: 250 scans = 1 s.

// HMI limits

IF HornStartBeep_s < REAL#0 THEN
    HornStartBeep_s := REAL#0;
ELSIF HornStartBeep_s > REAL#5 THEN
    HornStartBeep_s := REAL#5;
END_IF;

IF HornBeepLen_s < REAL#0 THEN
    HornBeepLen_s := REAL#0;
ELSIF HornBeepLen_s > REAL#2 THEN
    HornBeepLen_s := REAL#2;
END_IF;

IF HornBeepPeriod < REAL#0 THEN
    HornBeepPeriod := REAL#0;
ELSIF (HornBeepPeriod > REAL#0) AND (HornBeepPeriod < HornBeepLen_s + REAL#0.5) THEN
    HornBeepPeriod := HornBeepLen_s + REAL#0.5;    // always a pause between beeps
ELSIF HornBeepPeriod > REAL#30 THEN
    HornBeepPeriod := REAL#30;
END_IF;

IF HornStopBeep_s < REAL#0 THEN
    HornStopBeep_s := REAL#0;
ELSIF HornStopBeep_s > REAL#10 THEN
    HornStopBeep_s := REAL#10;
END_IF;

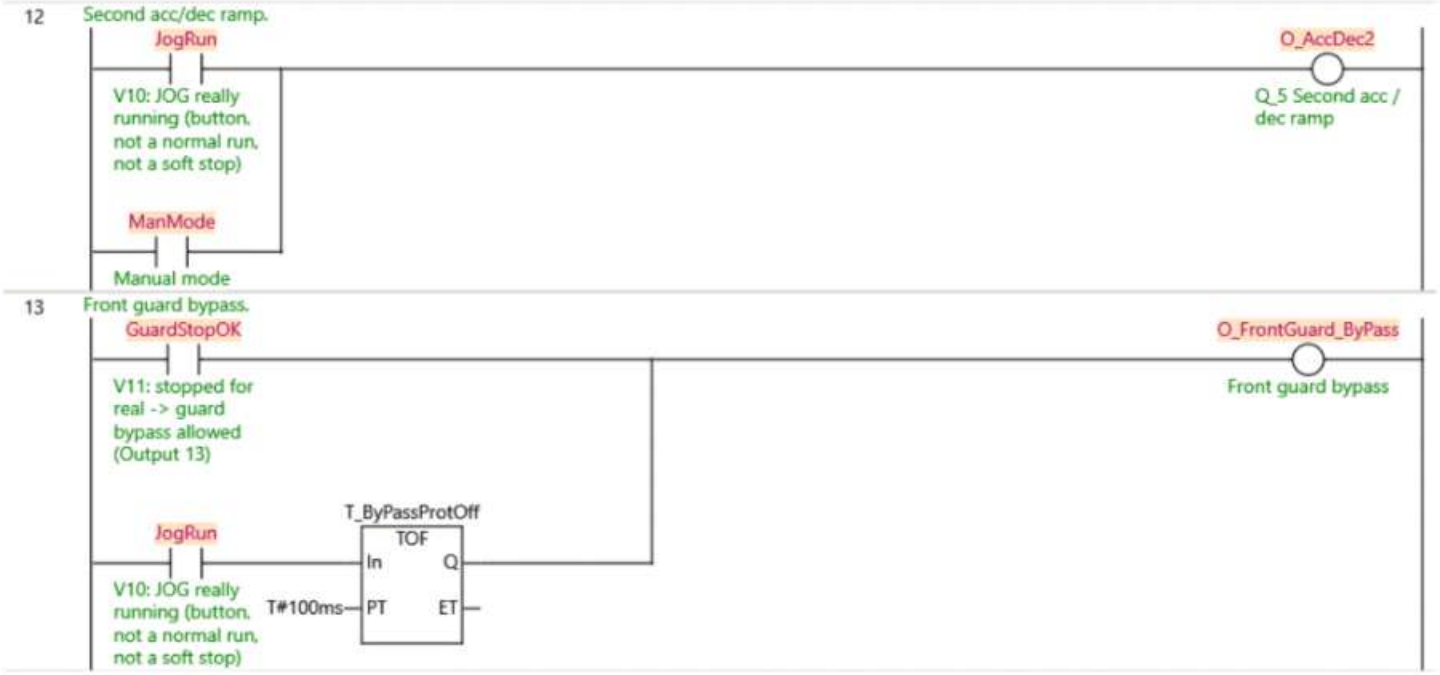
HornStartScans := REAL_TO_INT(HornStartBeep_s * REAL#250);
HornLenScans   := REAL_TO_INT(HornBeepLen_s * REAL#250);
HornPer        := REAL_TO_INT(HornBeepPeriod * REAL#250);

// Slow-down / final speed beeps

HornRun := StartMotor_Cmd AND SlowDown AND NOT Stop AND NOT JogRun;  // no horn while jogging

IF HornRun THEN
    IF NOT HornRunMem THEN
        HornTicks := 0:                    // slow-down starts
```

```
51      END_IF;
52
53      IF HornTicks < 30000 THEN
54          HornTicks := HornTicks + 1;
55      END_IF;
56
57      IF HornTicks <= HornStartScans THEN
58          HornSlowBeep := TRUE;                    // start beep
59          HornCnt := 0;
60      ELSIF (HornPer > 0) AND (HornLenScans > 0) THEN
61          HornCnt := HornCnt + 1;
62          IF HornCnt >= HornPer THEN
63              HornCnt := 0;
64          END_IF;
65
66          HornSlowBeep := (HornCnt >= HornPer - HornLenScans);   // beep at the end
    of each period
67      ELSE
68          HornSlowBeep := FALSE;                   // periodic beeps OFF
69      END_IF;
70  ELSE
71      HornTicks := 0;
72      HornCnt := 0;
73      HornSlowBeep := FALSE;
74  END_IF;
75
76  // End of a slow-down by a stop (auto stop, STOP button): one beep at standstill
77  // (Stop is set in Motor_Manager before StartMotor_Cmd drops on the next scan)
78
79  IF HornRunMem AND NOT HornRun AND (Stop OR NOT StartMotor_Cmd) THEN
80      HornStopPend := TRUE;
81  END_IF;
82
83  IF StartMotor_Cmd AND NOT HornStartMem THEN
84      HornStopPend := FALSE;                       // restarted: no stop beep
85      HornStopTicks := 0;
86  END_IF;
87
88  HornStartMem := StartMotor_Cmd;
89
90  IF HornStopPend AND (Velocity_M_min < REAL#2) THEN
91      HornStopPend := FALSE;
92      HornStopTicks := REAL_TO_INT(HornStopBeep_s * REAL#250);
93  END_IF;
94
95  IF HornStopTicks > 0 THEN
96      HornStopTicks := HornStopTicks - 1;
97  END_IF;
98
99  HornRunMem := HornRun;
100
101 // HMI test button: horn while pressed (machine stopped only)
102
103 O_AcousticSignal := (NOT I_FrontGuard_Closed AND NOT I_FrontGuard_Open AND
    Clock1s)
104                 OR HornSlowBeep OR (HornStopTicks > 0)
105                 OR (HornTest AND NOT StartMotor_Cmd);
```


12
Second acc/dec ramp.
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
ManMode
Manual mode
O_AccDec2
Q_5 Second acc / dec ramp
13
Front guard bypass.
GuardStopOK
V11: stopped for real -> guard bypass allowed (Output 13)
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
T_ByPassProtOff
TOF
In
Q
T#100ms
PT
ET
O_FrontGuard_ByPass
Front guard bypass

1-8-1-1-8.AutoStop_Control

0 Auto Stop Process 1-5

```
(*
=============================================================
====================
  AutoStop_Control  V10  -  PART 1 of 2 (rung 0): sections 1 - 5c
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  V10: JogRun (jog really running) instead of the raw jog button (5, 5b);
       long baseline also restarted if the encoder goes 2 m back (4b).
  V9 : RwDiamRev = DT35 averaged over exactly one revolution (2b), used by
       the thickness learning (removes the out-of-round wave), RwWave_mm.
     GSM popup: once per real new roll (empty 3 s, then > stop + 100 mm
       for 3 s); NEW PAPER closes it (section 10).
     paper thickness: long-baseline learning (D^2 growth / length) from the
       start point, LOCKED at RwLockDiam (300 mm): no more change, no
       paper-change detection, until NEW PAPER / GSM popup YES (4b).
  V8 : = V7 + V7c unwinder DIAMETER slow-down follows the speed (6c),
       V7e ManMode slow-down model as before V7 (section 6),
       V7f S-curve extra length in the slow-down (section 6)
  V7 : FEED-FORWARD speed loop (Motor_Manager rung B): reference = set speed
     x RwRatioD / diameter along a ramp in real web mm/s2 (DecelWeb /
     AccelWeb), RwRatioD adapted slowly -> no hunting on big rolls.
     Section 6: slow-down length = one ramp at DecelWeb (FFEnable), else
     web coarse ramp + trim tail; learning only as fallback.
  v6y: rewinder AUTO stop (RwStopEnable) limits the work speed to
     RwAutoMaxSpd (500 m/min), taken over at standstill only; section 6
     uses the limited speed.
  v6x: new parent roll on the unwinder -> GsmAsk (HMI popup "did you change
     the paper?"); GsmAskYes forgets the learned thickness.
  v6w: paper thickness window is LENGTH-based instead of 10 s, modulated by
     the diameter (RwWinGrow_mm 2.5 mm of growth, 4 m .. RwWinLen_m 30 m):
     same accuracy at every speed and roll size; stopped window = dropped.
     ALARM RESET also clears LearnFailCnt / StopFailCnt (alarm 16).
  v6v: BrakeHold released on the SIGNED web speed (Velocity_M_min < 5),
     so a web pulled back after the stop releases the T-one at once.
  v6u: stop learning: a stop shorter than the reaction distance or faster
     than RwDecelMax_mm is ignored (not a fault). Only a too slow stop
     (< RwDecelMin_mm) counts toward alarm 16.
  v6t: rewinder slow-down fallback while the paper thickness is not
     learned: RwSlowReq at RwTargetDiam - RwSlowFallbackMm (30 mm).
  v6s: paper change: a learning window more than RwMatChangePct (25 %) away
     from RwMat_K is ignored once, taken directly the 2nd time in a row
     (automatic). RwMatReset (HMI "NEW PAPER") forgets RwMat_K.
  v6p: section 5c: RwLearnReset clears RwStopDecelK, RwStopDecel and
     RwSlowDecelK (the v6f seed that refilled RwStopDecelK is removed).
  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
       UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
       UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
       same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
```

```
55          Replaces General rung 15 (one immediate sonar reading).
56      v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
57          rewinder, machine stopped -> after 2 s stable:
58          RwAxisDist := RwDist + RwCoreDiameter / 2. Status in RwTeachState.
59      v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6,
60          Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
61          so section 6 uses the raw pots in ManMode.
62      v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
63          normal stop until the web is almost stopped (< 5 m/min, max 15 s).
64          Before: T-one switched to "stop" with the run command, while the
65          rewinder was decelerating hard -> unwinder overran, web on the
66          floor (field 10/20/2026). Used in Output_Manager rung 2.
67      v6g: slow-down AND stop decel proportional to the rewinder diameter.
68          The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
69          decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
70          24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
71          Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
72          Live:    RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
73      v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
74          (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
75          braking resistor), so the PLC ramp (DecelCoarse) was too
76          optimistic: the stop came before final speed was reached.
77          Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
78          (learned) > DecelCoarse/DecelTrim model.
79      v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
80          RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
81          there is no learning fault. Alarm 16 uses GSM_LearnFault.
82      v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
83          TON ran ~104-108 ms -> speed ~4-8 % too high)
84         standstill = <= 2 counts / 100 ms (encoder jitter)
85         stop learning skipped after jog / manual / slow-button runs and
86           when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
87         work / final speed from the potentiometers in BOTH modes
88           (Speed_Display / LowSpd_Display are not m/min in ManMode)
89         slow-down length includes the DecelTrim tail inside the
90           SpdDelta_Percent band (Motor_Manager rungs 14/15)
91      v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
92          thickness RwThick_mm, plausibility limits for K and stop decel
93      v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
94          while RwMat_K is not learned yet)
95      v6 : consistent internal names, units in names.
96         Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
97                          RwSlowMargin_m, UwSlowMargin_m
98      v5 : max diameter protection (always active), sensor plausibility,
99         12SA5 master (section 9), alarms in section Alarms (ladder)
100     v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
101
        ================================================================
        ==================== *)
102
103 (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
104 (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
105 (*      Change 25 if the PrimaryTask period is changed.                  *)
106
107 TickCnt := TickCnt + 1;
108
109 IF TickCnt >= 25 THEN
110     TickCnt := 0;
111     Tick := TRUE;
```

```
112 ELSE
113     Tick := FALSE;
114 END_IF;
115
116 IF (Encoder_Pulse > 0) AND (EncoderWheel_Circumf > 0.0) THEN
117     MmPerPulse := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) * 4.0);
118 ELSE
119     MmPerPulse := 0.0;
120 END_IF;
121
122 (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ---------- *)
123
124 RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
125             * (RwDist10V - RwDist0V);
126
127 IF Tick THEN
128
129     IF NOT RwFiltInit THEN
130         RwDist := RwDistRaw_mm;
131         RwFiltInit := TRUE;
132     ELSE
133         RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
134     END_IF;
135 END_IF;
136
137 RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
138             OR (RwDist > RwAxisDist);
139
140 RwDiam := 2.0 * (RwAxisDist - RwDist);
141
142 IF RwDiam < 0.0 THEN
143     RwDiam := 0.0;
144 END_IF;
145
146 (* ---- 2b. V9: diameter averaged over exactly ONE revolution ---------- *)
147 (* The out-of-round roll gives a wave at the rotation frequency (seen in   *)
148 (* the 10/29 traces: 2.4 Hz at 444 m/min / 973 mm). Averaging the raw     *)
149 (* DT35 distance over exactly one revolution cancels it (and all its      *)
150 (* harmonics), whatever the phase -> no shaft position sensor needed:     *)
151 (* one revolution = PI x D of web on the encoder. Each scan is weighted   *)
152 (* by its encoder pulses = by roll ANGLE, so ramps and standstill do not  *)
153 (* bias it (stopped: nothing added). Used only by the thickness learning. *)
154 (* RwWave_mm = peak-peak diameter wave in the last revolution (HMI).      *)
155
156 RevDP := ABS(Encoder_Pos - RevPrevPos);
157 RevPrevPos := Encoder_Pos;
158
159 IF RwSensFault OR (MmPerPulse <= 0.0) OR (RwDiam < 50.0) OR (RevDP > 100000)
    THEN
160     RevW := 0.0;
161     RevSum := 0.0;
162     RwDiamRevOk := RwDiamRevOk AND NOT RwSensFault;
163 ELSIF RevDP > 0 THEN
164
165     IF RevW <= 0.0 THEN
166         RevMin := RwDistRaw_mm;
167         RevMax := RwDistRaw_mm;
168     END_IF;
```

```
169
170     RevSum := RevSum + RwDistRaw_mm * DINT_TO_REAL(RevDP);
171     RevW := RevW + DINT_TO_REAL(RevDP);
172
173     IF RwDistRaw_mm < RevMin THEN
174         RevMin := RwDistRaw_mm;
175     END_IF;
176
177     IF RwDistRaw_mm > RevMax THEN
178         RevMax := RwDistRaw_mm;
179     END_IF;
180
181     IF RevW >= 3.14159 * RwDiam / MmPerPulse THEN
182         RwDiamRev := 2.0 * (RwAxisDist - RevSum / RevW);
183         RwWave_mm := 2.0 * (RevMax - RevMin);
184         RwDiamRevOk := TRUE;
185         RevW := 0.0;
186         RevSum := 0.0;
187     END_IF;
188 END_IF;
189
190 (* v6g: diameter used by the decel model (last valid on sensor fault)      *)
191
192 IF NOT RwSensFault THEN
193     DecelDiam := RwDiam;
194 ELSE
195     DecelDiam := RwLastValidDiam;
196 END_IF;
197
198 IF DecelDiam < 100.0 THEN
199     DecelDiam := 100.0;                    (* core *)
200 END_IF;
201
202 (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
203
204 IF Tick THEN
205
206     IF NOT SpdInit THEN
207         EncPrev := Encoder_Pos;
208         SpdInit := TRUE;
209     END_IF;
210     SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
211     EncPrev := Encoder_Pos;
212 END_IF;
213
214 RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;        (* m/min *)
215
216 (* ---- 4a. v6s: NEW PAPER button: forget the learned thickness ---------- *)
217 (* RwMat_K = 0 -> the first valid 10 s window sets it directly.           *)
218 (* HMI momentary button: must be HELD 3 s, then the reset is done once and *)
219 (* the PLC clears the bit (HMI lamp goes off = done).                     *)
220
221 T_MatReset(In := RwMatReset, PT := T#3s);
222
223 IF T_MatReset.Q THEN
224     RwMat_K := 0.0;
225     LearnFailCnt := 0;
226     MatChangeCnt := 0;
227     RwMatLocked := FALSE;                  (* V9 *)
```

```
    LongActive := FALSE;
    GsmAsk := FALSE;                    (* V9: NEW PAPER answers the popup *)
    GsmArmed := FALSE;
    RwMatReset := FALSE;
END_IF;

(* ---- 4b. V9: long-baseline thickness learning + LOCK ---------- *)
(* From the start point (first run after NEW PAPER / GSM YES / power-on    *)
(* while not locked): K = (D^2 - D0^2) / (2 x pulses). The waving of the   *)
(* roll (+/-0.4 mm) only matters at the two ends, so the error falls with  *)
(* the growth (50 mm growth: ~1 %). After 20 mm growth it replaces the     *)
(* 30 m windows. At RwLockDiam (and >= RwLockMinGrow_mm growth) the value  *)
(* is LOCKED (Retain): windows and paper-change detection stop.            *)
(* RwLockDiam = 0 -> never locked (windows + long baseline as learning).   *)

IF Tick AND NOT RwMatLocked THEN

    IF LongActive AND (RwDiamRev < LongStartD - 30.0) AND RwDiamRevOk THEN
        LongActive := FALSE;            (* new core: restart *)
    END_IF;

    IF Encoder_Reset OR (Encoder_Pos < LongStartPos - 20000) THEN
        LongActive := FALSE;            (* metre counter reset / 2 m back *)
    END_IF;

    IF NOT LongActive AND O_MotorStart AND RwDiamRevOk
      AND (RwDiamRev > 0.0) THEN
        LongStartPos := Encoder_Pos;
        LongStartD := RwDiamRev;
        LongActive := TRUE;
    END_IF;

    LongValid := FALSE;

    IF LongActive AND RwDiamRevOk AND (MmPerPulse > 0.0) THEN
        LongGrow_mm := RwDiamRev - LongStartD;
        LongPls := DINT_TO_REAL(ABS(Encoder_Pos - LongStartPos));

        IF (LongGrow_mm >= 20.0) AND (LongPls > 0.0) THEN
            LongK := (RwDiamRev * RwDiamRev - LongStartD * LongStartD) / (2.0 *
LongPls);
            LongThick_mm := LongK * 3.14159 / (2.0 * MmPerPulse);

            IF (LongThick_mm >= RwThickMin_mm) AND (LongThick_mm <=
RwThickMax_mm) THEN
                LongValid := TRUE;
                RwMat_K := LongK;
                MatChangeCnt := 0;
                LearnFailCnt := 0;

                IF (RwLockDiam > 0.0) AND (RwDiamRev >= RwLockDiam)
                  AND (LongGrow_mm >= RwLockMinGrow_mm) THEN
                    RwMatLocked := TRUE;        (* solid from now on *)
                    LongActive := FALSE;
                END_IF;
            END_IF;
        END_IF;
    END_IF;
END_IF;
```

```
284 END_IF;
285
286 IF RwMatLocked THEN
287     LongValid := FALSE;
288     LearnFailCnt := 0;                    (* no thickness alarm *)
289 END_IF;
290
291 (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ---------- *)
292
293 IF Tick THEN
294     LearnWinTicks := LearnWinTicks + 1;
295
296     IF NOT LearnInit THEN
297         LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9 *)
298         LearnWinTicks := 0;
299         LearnWinRunning := O_MotorStart;
300         LearnInit := TRUE;
301     ELSE
302         LearnWinRunning := LearnWinRunning AND O_MotorStart;
303         (* v6w: LENGTH-based window, modulated by the diameter: closes    *)
304         (* when the roll should have grown RwWinGrow_mm (2.5 mm), so the  *)
305         (* accuracy vs DT35 noise is the same at every speed and size.    *)
306         (* Pulses = growth x diameter / K (K = learned RwMat_K).          *)
307         (* Limits: min 4 m, max RwWinLen_m (30 m). K not learned: max.    *)
308         (* Motor stopped during the window -> window dropped (no fault).  *)
309
310         IF MmPerPulse > 0.0 THEN
311
312             IF (RwMat_K > 0.0) AND (RwWinGrow_mm > 0.0) THEN
313                 LearnWinLen_Pls := RwWinGrow_mm * MAX(RwDiam, REAL#100.0) /
    RwMat_K;
314             ELSE
315                 LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
316             END_IF;
317
318             IF LearnWinLen_Pls < 4000.0 / MmPerPulse THEN
319                 LearnWinLen_Pls := 4000.0 / MmPerPulse;          (* 4 m *)
320             END_IF;
321
322             IF (RwWinLen_m > 0.0)
323               AND (LearnWinLen_Pls > RwWinLen_m * 1000.0 / MmPerPulse) THEN
324                 LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
325             END_IF;
326         ELSE
327             LearnWinLen_Pls := 300000.0;
328         END_IF;
329
330         IF (DINT_TO_REAL(ABS(Encoder_Pos - LearnWinStartPos)) >= LearnWinLen_Pls)
331           OR NOT LearnWinRunning THEN
332             LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
333             LearnWinDiamDelta := RwDiamRev - LearnWinStartDiam;  (* V9 *)
334             (* A window SHOULD learn: ran all the time, enough length,  *)
335             (* DT35 OK. Then it must give a plausible paper thickness. *)
336
337             IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
338               AND NOT RwSensFault
339               AND NOT RwMatLocked AND NOT LongValid     (* V9 *)
340               AND RwDiamRevOk AND (LearnWinStartDiam > 0.0) THEN
341                 LearnWinOk := FALSE:
```

```
342
343             IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
344                MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
345                        * ((RwDiamRev + LearnWinStartDiam) / 2.0);
346                (* K = 2 * thickness * mm-per-pulse / PI        *)
347                ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
348
349                IF (ThickWin_mm >= RwThickMin_mm)
350                  AND (ThickWin_mm <= RwThickMax_mm) THEN
351                   LearnWinOk := TRUE;
352
353                   IF RwMat_K <= 0.0 THEN
354                      RwMat_K := MatK_Window;
355                      MatChangeCnt := 0;
356                   ELSIF (RwMatChangePct > 0.0)
357                       AND (ABS(MatK_Window - RwMat_K)
358                           > RwMat_K * RwMatChangePct / 100.0) THEN
359                      (* v6s: far from the learned value: 1 window = ignored *)
360                      (* (outlier), 2 in a row = paper changed -> take it.  *)
361                      MatChangeCnt := MatChangeCnt + 1;
362
363                      IF MatChangeCnt >= 2 THEN
364                         RwMat_K := MatK_Window;
365                         MatChangeCnt := 0;
366                      END_IF;
367                   ELSE
368                      MatChangeCnt := 0;
369                      RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
370                   END_IF;
371                END_IF;
372             END_IF;
373
374             IF LearnWinOk THEN
375                LearnFailCnt := 0;
376             ELSIF LearnFailCnt < 1000 THEN
377                LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
378             END_IF;
379          END_IF;
380          LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9
    *)
381          LearnWinTicks := 0;
382          LearnWinRunning := O_MotorStart;
383       END_IF;
384    END_IF;
385 END_IF;
386
387 (* ---- 5. Learn real stop deceleration on every stop ---------------------- *)
388 (* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
389 (*      -> its stop must not be learned. Latched for the whole run because  *)
390 (*      the jog button is already released on the stop edge.                *)
391
392 IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
393    RunWasJog := FALSE;
394 END_IF;
395
396 IF O_MotorStart AND (ManMode OR JogRun) THEN
397    RunWasJog := TRUE;
398 END_IF;
399
```

```
IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
   StopMeasActive := TRUE;
   StopStartPos := Encoder_Pos;
   StopStartSpd_Pps := SpdAct_Pps;
   StopStartDiam := DecelDiam;
   StandstillTicks := 0;
   (* STOP button and automatic stop take the same path (drive DI1 and    *)
   (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),     *)
   (* so both are learned.                                                *)
   StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
END_IF;

MotorRunMem := O_MotorStart;

IF O_MotorStart THEN
   StopMeasActive := FALSE;                (* restarted: abort *)
ELSIF StopMeasActive AND Tick THEN

   IF NOT ControlVoltage THEN
      StopLearnValid := FALSE;             (* E-stop during the stop *)
   END_IF;

   IF SpdAct_Pps <= 20.0 THEN              (* <= 2 counts / 100 ms *)
      StandstillTicks := StandstillTicks + 1;
   ELSE
      StandstillTicks := 0;
   END_IF;

   IF StandstillTicks >= 3 THEN            (* standstill 300 ms *)
      StopMeasActive := FALSE;
      StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));

      IF RwLearnStop AND StopLearnValid
        AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
         (* This stop SHOULD learn: result must be plausible      *)
         (* v6u: a stop shorter than the reaction distance, or    *)
         (* faster than RwDecelMax_mm, is NOT a fault: the web    *)
         (* stopped within a few mm, nothing to learn (trace      *)
         (* 10/28/2026: 23 m/min -> stopped in 102 pulses = 10 mm). *)
         (* Only a too SLOW stop (< RwDecelMin_mm) counts as fail.  *)
         StopLearnOk := FALSE;

         IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
            StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
                        / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
            StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)

            IF (StopDecelMeas_mm >= RwDecelMin_mm)
              AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
               StopLearnOk := TRUE;
               (* v6g: learn per mm of diameter *)
               DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) /
2.0);

               IF RwStopDecelK <= 0.0 THEN
                  RwStopDecelK := DecelK_Meas;
               ELSE
                  RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas -
RwStopDecelK);
```

```
                RwStopDecelK);
                END_IF;
                StopFailCnt := 0;
            ELSIF StopDecelMeas_mm < RwDecelMin_mm THEN

                IF StopFailCnt < 1000 THEN
                    StopFailCnt := StopFailCnt + 1;   (* stop too long *)
                END_IF;
            END_IF;                              (* too fast: ignored *)
        END_IF;                                  (* too short: ignored *)
    END_IF;
  END_IF;
END_IF;

(* ---- 5b. v6f: learn real slow-down deceleration ----------------------- *)
(* From the SlowDown edge (at speed) until final speed is reached, or     *)
(* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
(* Not learned in jog / manual / slow button (2nd ramp).                  *)

IF SlowDown AND NOT SlowMem AND O_MotorStart
  AND NOT ManMode AND NOT JogRun
  AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
   SlowMeasActive := TRUE;
   SlowStartPos := Encoder_Pos;
   SlowStartSpd_Pps := SpdAct_Pps;
   SlowStartDiam := DecelDiam;
END_IF;
SlowMem := SlowDown;

IF SlowMeasActive THEN

  IF NOT SlowDown OR ManMode OR JogRun THEN
     SlowMeasActive := FALSE;                (* aborted *)
  ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
                OR NOT O_MotorStart) THEN      (* final reached or stopped *)
     SlowMeasActive := FALSE;
     SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));

     IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
       AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
        SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
                           - SpdAct_Pps * SpdAct_Pps)
                          / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)

        IF (SlowDecelMeas_mm >= RwDecelMin_mm)
          AND (SlowDecelMeas_mm >= RwSlowLearnMin_mm)    (* Fix 11/11 *)
          AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
           (* v6g: learn per mm of diameter *)
           DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) /
2.0);

           IF RwSlowDecelK <= 0.0 THEN
              RwSlowDecelK := DecelK_Meas;
           ELSE
              RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas -
RwSlowDecelK);
           END_IF;
        END_IF;
     END_IF;
```

```
513     END_IF;
514 END_IF;
515
516 (* ---- 5c. v6g: live decels = learned K x current diameter ---------- *)
517 (* v6p: the one-time v6f seed is removed (it re-filled RwStopDecelK from    *)
518 (* RwStopDecel, so the learning could never be reset).                     *)
519 (* RwLearnReset (HMI / Watch tab, PLC resets it): clear the learned decels. *)
520 (* Both are re-learned at the next auto slow-down / stop >= RwMinLearnSpd.  *)
521
522 IF RwLearnReset THEN
523     RwStopDecelK := 0.0;
524     RwStopDecel := 0.0;
525     RwSlowDecelK := 0.0;
526     StopFailCnt := 0;
527     RwLearnReset := FALSE;
528 END_IF;
529
530 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
531     RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
532 END_IF;
533
534 IF RwSlowDecelK > 0.0 THEN
535     RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
536 ELSE
537     RwSlowDecelLrn := 0.0;
538 END_IF;
```

1 Auto Stop Process 6-14

```
(*
 ================================================================
 ====================
   AutoStop_Control  V12  -  PART 2 of 2 (rung 1): sections 6 - 14c
   V12: END-OF-ROLL STOP RAMP: auto / diameter stops ramp down at
 FinalStopDecel
        (soft stop mechanism) instead of the drive DEC ramp (3 s = 20 Hz/s,
        up to ~8 m/s2 of web on a big roll); stop lengths use it (6, 11b).
   V11: 3-state unwinder display + stop command at UwStop_at_D_Val (section 8).
   (Sysmac: an inline ST rung holds max 1000 rows -> split in 2 rungs.
    Rung 0 = part 1 runs first, then this rung, every scan.)

 ================================================================
 ==================== *)

(* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)
(* Stop length uses the ACTUAL speed (precision)                          *)

IF RwStopDecel > 0.0 THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
               + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
ELSE
   StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
END_IF;

(* V12: end-of-roll stop ramp at FinalStopDecel (web mm/s2) + S-curve    *)

IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode AND (MmPerPulse >
 0.0) THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
               + (SpdAct_Pps * SpdAct_Pps) / (2.0 * FinalStopDecel / MmPerPulse)
               + SpdAct_Pps * 0.5 * SCurveTime;
END_IF;

StopLen_Pls := StopLen_Pls * RwTrim;

(* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):              *)
(* stable value, shown to the operator even before start                  *)

SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;

IF MmPerPulse > 0.0 THEN
   (* v6f: manual > learned > PLC ramp model                              *)
   (* V7: manual > feed-forward (one ramp at DecelWeb, real web mm/s2)  *)
   (*     > old loop with web ramps (coarse DecelWeb + trim tail)       *)
   (*     > learned > old PLC ramp model                                 *)

   IF RwSlowDecel > 0.0 THEN
      SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF FFEnable AND (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT
 ManMode THEN
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT ManMode THEN  (* V7e
 *)
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := FALSE;
```

```
51    ELSIF RwSlowDecelLrn > 0.0 THEN
52       SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
53       SlowOneRamp := TRUE;
54    ELSE
55       SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
56       SlowOneRamp := FALSE;
57    END_IF;
58
59    (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
60
61    IF (DecelTrim > 0.0) AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0) THEN
62       (* V7: trim ramp (reference units) converted to the real web decel *)
63       TrimDecel_Pps2 := DecelTrim * 1000000.0 / (0.06 * WebRatio * Vel_Max_mms)
64                         / MmPerPulse;
65    ELSIF DecelTrim > 0.0 THEN
66       TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
67    ELSE
68       TrimDecel_Pps2 := SlowDecel_Pps2;
69    END_IF;
70
71    SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;
72
73    (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
74    (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
75
76    IF (Input_Ratio > 0.0) AND ManMode THEN
77       (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
78       SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
79       SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
80    ELSIF Input_Ratio > 0.0 THEN
81       SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
82       SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
83    ELSE
84       SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
85       SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
86    END_IF;
87
88    (* v6y: rewinder AUTO stop -> work speed limited to RwAutoMaxSpdAct *)
89    (* (limit + value taken over only at standstill, Motor_Manager rung 5) *)
90
91    IF RwAutoLimAct AND NOT ManMode
92       AND (SpdWork_Pps > RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse) THEN
93       SpdWork_Pps := RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse;
94    END_IF;
95
96    SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
97
98    IF RwStopDecel > 0.0 THEN
99       StopLenFinal_Pls := SpdFinal_Pps * RwTreact
100                         + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
101   ELSE
102      StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
103   END_IF;
104
105   (* V12: final stop from the final speed on the end-of-roll ramp *)
```

```
106
107   IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode THEN
108      StopLenFinal_Pls := SpdFinal_Pps * RwTreact
109                        + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * FinalStopDecel /
      MmPerPulse)
110                        + SpdFinal_Pps * 0.5 * SCurveTime;
111   END_IF;
112
113   SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
114
115   IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
116
117      IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
118         (* manual or learned decel: one constant ramp *)
119         SlowLenBase_Pls := SlowLenBase_Pls
120            + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
121             / (2.0 * SlowDecel_Pps2);
122      ELSE
123         (* auto: coarse ramp down to the band, trim ramp inside it *)
124         SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
125
126         IF SpdRef_Pps > SpdKnee_Pps THEN
127            SlowLenBase_Pls := SlowLenBase_Pls
128               + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
129                / (2.0 * SlowDecel_Pps2)
130               + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
131                / (2.0 * TrimDecel_Pps2);
132         ELSE
133            SlowLenBase_Pls := SlowLenBase_Pls
134               + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
135                / (2.0 * TrimDecel_Pps2);
136         END_IF;
137      END_IF;
138   END_IF;
139
140   (* V7f: S-curve rounding: the slow-down ends SCurveTime later ->      *)
141   (*      extra length = (SpdRef - SpdFinal) / 2 x SCurveTime          *)
142
143   IF FFEnable AND NOT ManMode AND (RwSlowDecel <= 0.0) AND (DecelWeb >
      0.0)
144      AND (WebRatio > 0.0) AND (SCurveTime > 0.0)
145      AND (SpdRef_Pps > SpdFinal_Pps) THEN
146      SlowLenBase_Pls := SlowLenBase_Pls
147                        + (SpdRef_Pps - SpdFinal_Pps) * 0.5 * SCurveTime;
148   END_IF;
149
150   RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;
151
152   IF UwSlowMargin_m > 0.0 THEN
153      UwMarginUsed_m := UwSlowMargin_m;
154   ELSE
155      UwMarginUsed_m := RwSlowMargin_m;
156   END_IF;
157
158   UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
159 END_IF;
160
161 IF RwDiam >= RwTargetDiam THEN
```

```
163 ELSIF RwMat_K > 0.0 THEN
164     RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
165                     / (2.0 * RwMat_K);
166 ELSE
167     RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
168 END_IF;
169
170 RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
171 RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
172 RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;
173
174 (* ---- 6b. Live operator displays ---------------------------------------- *)
175 (* Rewinder: diameter where slow-down / stop command will happen          *)
176
177 IF RwMat_K > 0.0 THEN
178     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
179
180     IF SqrtArg > 0.0 THEN
181         RwSlow_at_D_Val := SQRT(SqrtArg);
182     ELSE
183         RwSlow_at_D_Val := 0.0;
184     END_IF;
185
186     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;
187
188     IF SqrtArg > 0.0 THEN
189         RwStop_at_D_Val := SQRT(SqrtArg);
190     ELSE
191         RwStop_at_D_Val := 0.0;
192     END_IF;
193 ELSE
194     (* v6t: thickness not learned: fixed slow-down point before the target *)
195
196     IF RwUseSlow AND (RwSlowFallbackMm > 0.0) THEN
197         RwSlow_at_D_Val := RwTargetDiam - RwSlowFallbackMm;
198     ELSE
199         RwSlow_at_D_Val := RwTargetDiam;
200     END_IF;
201
202     RwStop_at_D_Val := RwTargetDiam;
203 END_IF;
204
205 (* Unwinder: meter count where the slow-down will start                   *)
206
207 IF MmPerPulse > 0.0 THEN
208     UwSlow_at_M_Val := UwStop_at_M_Val - UwSlowLen_Pls * MmPerPulse / 1000.0;
209 ELSE
210     UwSlow_at_M_Val := UwStop_at_M_Val;
211 END_IF;
212
213 IF UwSlow_at_M_Val < 0.0 THEN
214     UwSlow_at_M_Val := 0.0;
215 END_IF;
216
217 (* Unwinder slow-down request, latched                                    *)
218
219 IF (Current_Metre < UwSlow_at_M_Val - 50.0) OR (UwStop_at_M_Val <= 0.0) OR
    CntResetPend THEN
```

```
220     UwSlow_at_M_Req := FALSE;                    (* counter reset / longer roll *)
221   END_IF;
222
223   IF (UwStop_at_M_Val > 0.0) AND (Current_Metre >= UwSlow_at_M_Val) AND NOT
      CntResetPend THEN
224     UwSlow_at_M_Req := TRUE;
225   END_IF;
226
227   (* ---- 6c. V7c: unwinder DIAMETER slow-down follows the speed ---------- *)
228   (* Works without AutoStop (only Measure_ON, like General rung 16).       *)
229   (* Paper left above the stop diameter, in pulses:                        *)
230   (*     (UwDiam_Act^2 - UwStopDiam_Limit^2) / (2 x RwMat_K)               *)
231   (* (same paper as the rewinder -> same learned K). Slow down when it is  *)
232   (* shorter than UwSlowLen_Pls = decel from the speed (DecelWeb) + final   *)
233   (* stop + UwSlowMargin_m. Solved for the diameter:                       *)
234   (*     UwSlowDiamCalc = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x
      UwSlowLen_Pls)   *)
235   (* K not learned / UwSlow_at_D_Req OFF -> fixed UwSlow_at_D_Val as before. *)
236   (* Latched: once slowed down, the point is frozen (+10 mm hysteresis), so  *)
237   (* the lower speed does not move it back and speed up again. Released by   *)
238   (* a new roll (diameter 30 mm above the frozen point).                   *)
239
240   IF UwSlow_at_D_Req AND (RwMat_K > 0.0) AND (UwStopDiam_Limit > 0.0)
241     AND (UwSlowLen_Pls > 0.0) THEN
242     SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
      UwSlowLen_Pls;
243     UwSlowDiamCalc := SQRT(SqrtArg);            (* same pattern as 6b *)
244   ELSE
245     UwSlowDiamCalc := UwSlow_at_D_Val;
246   END_IF;
247
248   IF UwSlowDiamCalc < UwStopDiam_Limit + 10.0 THEN
249     UwSlowDiamCalc := UwStopDiam_Limit + 10.0;
250   ELSIF UwSlowDiamCalc > 1500.0 THEN
251     UwSlowDiamCalc := 1500.0;
252   END_IF;
253
254   IF UwSlowDiamLatch AND ((UwDiam_Act > UwDiamAtLatch + 30.0)
255     OR (NOT StartMotor_Cmd AND (UwDiam_Act > UwSlowDiamCalc + 30.0))) THEN
256     UwSlowDiamLatch := FALSE;                   (* new roll / point moved, Fix 11/11 *)
257   END_IF;
258
259   IF NOT UwSlowDiamLatch THEN
260     UwSlowDiamAct := UwSlowDiamCalc;            (* live point, HMI *)
261
262     IF Measure_ON AND StartMotor_Cmd AND (UwDiam_Act > 0.0)
263       AND (UwDiam_Act <= UwSlowDiamCalc) THEN
264       UwSlowDiamLatch := TRUE;
265       UwDiamAtLatch := UwDiam_Act;              (* diameter when latched *)
266       UwSlowDiamAct := UwSlowDiamCalc + 10.0;   (* frozen + hysteresis *)
267     END_IF;
268   END_IF;
269
270   (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
271
272   IF NOT RwStopEnable
273     OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
274     RwStop_at_D_Req := FALSE;
275     RwSlow_at_D_Req := FALSE;
```

```
276 END_IF;
277
278 IF RwStopEnable THEN
279
280   IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
281     RwStop_at_D_Req := TRUE;
282   END_IF;
283
284   IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
285     RwSlow_at_D_Req := TRUE;
286   END_IF;
287
288   (* v6t: thickness not learned (RwMat_K = 0): slow down at a fixed     *)
289   (* distance before the target instead of stopping from full speed.    *)
290
291   IF RwUseSlow AND (RwMat_K <= 0.0) AND (RwSlowFallbackMm > 0.0)
292     AND NOT RwSensFault
293     AND (RwDiam >= RwTargetDiam - RwSlowFallbackMm) THEN
294     RwSlow_at_D_Req := TRUE;
295   END_IF;
296 END_IF;
297
298 (* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ---------- *)
299  // ---- Refuse AUTO STOP when its target is already reached (HMI warning 21) ----
300  // AutoEnWhy: 1 = unwinder metres already reached, 2 = rewinder roll >= target
301
302 IF I_AutoStop_OS AND RwStopEnable AND NOT RwArmMem AND NOT RwSensFault
303   AND (RwTargetDiam > REAL#0.0) AND (RwDiam >= RwTargetDiam -
    RwResetHyst) THEN
304   RwStopEnable := FALSE;                    // refused
305   AutoEnRefused := TRUE;
306   AutoEnWhy := 2;
307 END_IF;
308
309 IF I_AutoStop_OS AND UwStopEnable AND NOT UwArmMem AND NOT
    CntResetPend
310   AND (UwStop_at_M_Val > REAL#0.0) AND (Current_Metre >= UwStop_at_M_Val)
    THEN
311   UwStopEnable := FALSE;                    // refused
312   AutoEnRefused := TRUE;
313   AutoEnWhy := 1;
314 END_IF;
315
316 IF UwStopEnable AND NOT UwEnMem THEN            (* unwinder AUTO switched
    ON *)
317   RwStopEnable := FALSE;
318 END_IF;
319
320 IF RwStopEnable AND NOT RwEnMem THEN            (* rewinder AUTO switched
    ON *)
321   UwStopEnable := FALSE;
322 END_IF;
323
324  T_AutoEnRef(In := AutoEnRefused, PT := T#5s);      // warning shown 5 s
325
326 IF T_AutoEnRef.Q OR I_RsetAlarm OR Alarm_Reset THEN
327   AutoEnRefused := FALSE;
328 END_IF;
329
```

```
330  RwArmMem := I_AutoStop_OS AND RwStopEnable;       // armed = selector 12SA5
     AND enable
331  UwArmMem := I_AutoStop_OS AND UwStopEnable;
332  UwEnMem := UwStopEnable;
333  RwEnMem := RwStopEnable;
334
335  IF RwStopEnable THEN
336      SlowAtDisp := RwSlow_at_D_Val;                (* mm *)
337      SlowAtMode := 2;
338  ELSIF UwStopEnable THEN
339      SlowAtDisp := UwSlow_at_M_Val;                (* m  *)
340      SlowAtMode := 1;
341  ELSE
342      SlowAtDisp := 0.0;
343      SlowAtMode := 0;                          (* hide display *)
344  END_IF;
345
346  SlowAtShow := (SlowAtMode <> 0);
347
348  (* V11: unwinder column, same place, 3 states:                          *)
349  (*   AUTO STOP UNWINDER (12SA5 + unwinder AUTO) -> STOP LENGTH [m]      *)
350  (*   normal run, before the slow-down          -> SLOW AT Ø [mm]        *)
351  (*   slow-down reached (decel / final speed)   -> STOP AT Ø [mm]        *)
352  (* V11: stop by diameter EARLIER, so the web is stopped AT UwStopDiam_Limit:  *)
353  (* the stop command comes at UwStop_at_D_Val = UwStopDiam_Limit + the
     diameter   *)
354  (* of the paper unwound during the stop from the ACTUAL speed           *)
355  (* (StopLen_Pls, section 6, learned stop decel):                        *)
356  (*   UwStop_at_D_Val = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x StopLen_Pls)
     *)
357  (* General rung 18 (UwStopByDiam) compares with UwStop_at_D_Val.        *)
358  (* K not learned -> UwStopDiam_Limit (as before). Max UwStopDiam_Limit + 100
     mm.    *)
359
360  UwStopLenShow := I_AutoStop_OS AND (UwStopEnable OR (AutoEnRefused AND
     (AutoEnWhy = 1)));
361  UwSlowDiamShow := NOT UwStopLenShow AND Measure_ON AND NOT
     UwSlowDiamLatch;
362  UwStopDiamShow := NOT UwStopLenShow AND Measure_ON AND
     UwSlowDiamLatch;
363
364  IF (RwMat_K > 0.0) AND (StopLen_Pls > 0.0) THEN
365      SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
     StopLen_Pls;
366      UwStop_at_D_Val := SQRT(SqrtArg);
367  ELSE
368      UwStop_at_D_Val := UwStopDiam_Limit;
369  END_IF;
370
371  IF UwStop_at_D_Val > UwStopDiam_Limit + 100.0 THEN
372      UwStop_at_D_Val := UwStopDiam_Limit + 100.0;
373  ELSIF UwStop_at_D_Val < UwStopDiam_Limit THEN
374      UwStop_at_D_Val := UwStopDiam_Limit;
375  END_IF;
376
377  (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
378
379  AutoSlowCmd := I_AutoStop_OS AND
380              ((UwStopEnable AND UwSlow_at_M_Req) OR (RwStopEnable AND
```

```
    RwSlow_at_D_Req));
381
382 AutoStopCmd := I_AutoStop_OS AND
383             ((UwStopEnable AND (UwStop_at_M_Val > 0.0)
384               AND (Current_Metre >= UwStop_at_M_Val) AND NOT CntResetPend)
385              OR (RwStopEnable AND RwStop_at_D_Req));
386
387 (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
388
389 IF UwDiam_Act > (UwStopDiam_Limit + UwEmptyHyst) THEN
390     UnWinder_Empty := FALSE;                    (* new roll loaded *)
391 END_IF;
392
393 IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwStopDiam_Limit) THEN
394     UnWinder_Empty := TRUE;                     (* roll empty *)
395 END_IF;
396
397 (* v6x: new parent roll loaded (empty -> not empty) -> HMI popup        *)
398 (* "DID YOU CHANGE THE PAPER (GSM)?". YES = forget the learned thickness *)
399 (* (same as NEW PAPER), NO = the HMI resets GsmAsk.                     *)
400 (* V9: ask only ONCE per REAL new roll: the roll must have been empty for *)
401 (* 3 s (arms the question), then the new roll must read more than       *)
402 (* UwStopDiam_Limit + 100 mm for 3 s. The sonar bouncing around
    UwStopDiam_Limit    *)
403 (* (empty core, hands, arms) no longer re-opens the popup.              *)
404
405 IF Tick THEN
406
407     IF UnWinder_Empty THEN
408
409         IF UwEmptyTicks < 1000 THEN
410             UwEmptyTicks := UwEmptyTicks + 1;
411         END_IF;
412
413         IF UwEmptyTicks >= 30 THEN
414             GsmArmed := TRUE;
415         END_IF;
416         UwFullTicks := 0;
417     ELSE
418         UwEmptyTicks := 0;
419
420         IF GsmArmed AND Measure_ON AND (UwDiam_Act > UwStopDiam_Limit +
    100.0) THEN
421             UwFullTicks := UwFullTicks + 1;
422
423             IF UwFullTicks >= 30 THEN
424                 GsmAsk := TRUE;
425                 GsmArmed := FALSE;
426                 UwFullTicks := 0;
427             END_IF;
428         ELSE
429             UwFullTicks := 0;
430         END_IF;
431     END_IF;
432 END_IF;
433
434 UwEmptyMem := UnWinder_Empty;
435
436 IF GsmAskYes THEN
```

```
437     RwMat_K := 0.0;
438     RwMatLocked := FALSE;                   (* V9 *)
439     LongActive := FALSE;
440     LearnFailCnt := 0;
441     MatChangeCnt := 0;
442     GsmAskYes := FALSE;
443     GsmAsk := FALSE;
444   END_IF;
445
446   (* V10: HMI popup "UNWINDER DIAMETER SENSOR SONAR IS OFF ! DO YOU WANT
      TO  *)
447   (* TURN IT ON ?" - only at standstill: at power-up and after every stop    *)
448   (* while Measure_ON = FALSE. YES (SonarAskYes, PLC resets it) = sonar ON;  *)
449   (* NO = the HMI resets SonarAsk (asked again after the next stop).        *)
450
451   IF Measure_ON THEN
452     SonarAsk := FALSE;                    (* sonar ON: no popup *)
453   ELSIF NOT StartMotor_Cmd AND (SonarRunMem OR NOT SonarInit) THEN
454     SonarAsk := TRUE;                     (* stopped / power-up *)
455   END_IF;
456
457   SonarInit := TRUE;
458   SonarRunMem := StartMotor_Cmd;
459
460   IF StartMotor_Cmd THEN
461     SonarAsk := FALSE;                    (* never while running *)
462   END_IF;
463
464   IF SonarAskYes THEN
465     Measure_ON := TRUE;
466     SonarAsk := FALSE;
467     SonarAskYes := FALSE;
468   END_IF;
469
470   (* ---- 11. Rewinder max diameter protection - ALWAYS active ---------- *)
471   (* Independent of selector 12SA5 and of the AUTO buttons.               *)
472   (* Mechanical limit 680 mm radius = 1360 mm                            *)
473
474   IF NOT RwSensFault THEN
475     RwLastValidDiam := RwDiam;
476   END_IF;
477
478   IF RwDiam >= RwMaxDiam THEN
479     RwRemainMax_Pls := 0.0;
480   ELSIF RwMat_K > 0.0 THEN
481     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
482                        / (2.0 * RwMat_K);
483   ELSE
484     RwRemainMax_Pls := 1.0E9;             (* K not learned: plain limit *)
485   END_IF;
486
487   (* Display: diameter where the max-protection slow-down will start       *)
488
489   IF RwMat_K > 0.0 THEN
490     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
491
492     IF SqrtArg > 0.0 THEN
493       RwMaxSlowAtDiam := SQRT(SqrtArg);
494     ELSE
```

```
495         RwMaxSlowAtDiam := 0.0;
496     END_IF;
497 ELSIF RwMaxSlowFallback > 0.0 THEN
498     RwMaxSlowAtDiam := RwMaxSlowFallback;          (* fallback in use *)
499 ELSE
500     RwMaxSlowAtDiam := RwMaxDiam;
501 END_IF;
502
503 (* Reset only after the roll is removed (diameter well below the limit)    *)
504
505 IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
506     RwMaxStop := FALSE;
507     RwMaxSlow := FALSE;
508 END_IF;
509
510 IF NOT RwSensFault THEN
511
512     IF RwRemainMax_Pls <= StopLen_Pls THEN
513         RwMaxStop := TRUE;                         (* speed-compensated stop *)
514     END_IF;
515
516     IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
517         RwMaxSlow := TRUE;                         (* warning slow-down + horn *)
518     END_IF;
519
520     IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
521       AND (RwDiam >= RwMaxSlowFallback) THEN
522         RwMaxSlow := TRUE;                         (* fallback while K not learned *)
523     END_IF;
524
525 ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
    100.0) THEN
526     RwMaxStop := TRUE;                             (* roll grew into the DT35 near zone *)
527 END_IF;
528
529 (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)      *)
530
531 AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
532 AutoStopCmd := AutoStopCmd OR RwMaxStop;
533
534 (* ---- 11b. V12: END-OF-ROLL STOP RAMP ---------------------------------- *)
535 (* Normal end-of-roll stops (AutoStopCmd: unwinder metres, rewinder target,*)
536 (* rewinder max diameter; UwStopByDiam: unwinder diameter) no longer drop  *)
537 (* the run command at once (drive DEC ramp 3 s = 20 Hz/s -> up to 8 m/s2  *)
538 (* of web on a big roll, hard on the rewinder gearbox). With the feed-    *)
539 (* forward in AUTO they start the SOFT STOP at FinalStopDecel (General    *)
540 (* rung 10); the run command drops below SoftStopEndSpd. START cannot     *)
541 (* cancel it. The stop lengths (section 6) use the same ramp, so the web  *)
542 (* still stops at the target / UwStopDiam_Limit.                          *)
543 (* Immediate stop (EndStopNow -> Motor_Manager rung 4 Stop):              *)
544 (*   ramp not usable (FinalStopDecel 0, FFEnable OFF, ManMode, JOG, the   *)
545 (*   motor stopped = start blocked as before), or safety net: unwinder    *)
546 (*   10 mm below UwStopDiam_Limit / rewinder at RwMaxDiam during the ramp.*)
547
548 IF FinalStopDecel < 0.0 THEN
549     FinalStopDecel := 0.0;
550 ELSIF (FinalStopDecel > 0.0) AND (FinalStopDecel < 50.0) THEN
551     FinalStopDecel := 50.0;
552 ELSIF FinalStopDecel > 2000.0 THEN
```

```
    FinalStopDecel := 2000.0;
END_IF;

EndStopReq := AutoStopCmd OR (Measure_ON AND UwStopByDiam);
EndRampOK := (FinalStopDecel > 0.0) AND FFActive AND StartMotor_Cmd;

IF EndStopReq AND EndRampOK THEN
    FinalStopRamp := TRUE;                    (* -> soft stop, General 10 *)
END_IF;

IF NOT StartMotor_Cmd THEN
    FinalStopRamp := FALSE;
END_IF;

EndRampHard := FinalStopRamp
            AND ((Measure_ON AND (UwDiam_Act > 0.0)
                AND (UwDiam_Act < UwStopDiam_Limit - 10.0))
                OR ((RwDiam >= RwMaxDiam) AND NOT RwSensFault));

EndStopNow := (EndStopReq AND NOT FinalStopRamp) OR EndRampHard;

// ---- 11c. Auto stop DONE (HMI warning 20) ----
// AutoStopWhy: 1 unwinder metres, 2 rewinder diameter, 4 rewinder MAX, 8
unwinder diameter

IF StartMotor_Cmd AND NOT AS_RunMem THEN          // new START: clear
    AutoStopDone := FALSE;
    AutoStopSeen := FALSE;
    AutoStopWhy := 0;
END_IF;

IF StartMotor_Cmd AND EndStopReq AND NOT AutoStopSeen AND NOT JogRun
THEN
    AutoStopSeen := TRUE;                     // auto stop started while running

    IF RwMaxStop THEN
        AutoStopWhy := 4;                     // rewinder MAX (protection, highest)
    ELSIF I_AutoStop_OS AND RwStopEnable AND RwStop_at_D_Req THEN
        AutoStopWhy := 2;                     // rewinder diameter reached
    ELSIF Measure_ON AND UwStopByDiam THEN
        AutoStopWhy := 8;                     // unwinder diameter (sonar)
    ELSE
        AutoStopWhy := 1;                     // unwinder metres
    END_IF;
END_IF;

IF AS_RunMem AND NOT StartMotor_Cmd AND AutoStopSeen THEN
    AutoStopDone := TRUE;                     // stopped by the auto stop
END_IF;

IF I_RsetAlarm OR Alarm_Reset THEN
    AutoStopDone := FALSE;
END_IF;

T_AutoStopDisp(In := AutoStopDone, PT := T#8s);   // show the warning 8 s

IF T_AutoStopDisp.Q THEN
    AutoStopDone := FALSE;
END_IF;
```

```
610
611  AS_RunMem := StartMotor_Cmd;
612
613 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
614 (* Learned paper thickness, for display / plausibility check              *)
615
616 IF MmPerPulse > 0.0 THEN
617     RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
618  ELSE
619     RwThick_mm := 0.0;
620  END_IF;
621
622 (* Fault: too many consecutive windows / stops that should have           *)
623 (* learned but did not. Clears by itself on the next good result.        *)
624 (* v6w: ALARM RESET (panel button or HMI) also clears the fail counters, *)
625 (* so alarm 16 resets in one press. It comes back only if the learning  *)
626 (* fails again RwLearnFailMax / RwStopFailMax times in a row.            *)
627
628 IF I_RsetAlarm OR Alarm_Reset THEN
629     LearnFailCnt := 0;
630     StopFailCnt := 0;
631  END_IF;
632
633  GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
634             OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)
635
636 (* v6e: learning status for the HMI. FALSE until learned.                 *)
637
638  GSM_Learned_OK := (RwMat_K > 0.0)
639                AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
640                AND NOT GSM_LearnFault;
641
642 (* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
643 (* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
644 (* After the stop: stays ON until the web is below 5 m/min, after 15 s,    *)
645 (* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
646 (* this section), so O_Brake never drops for one scan.                    *)
647 (* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold              *)
648
649 IF O_MotorStart THEN
650     BrakeHold := Brake_Cmd AND ControlVoltage;
651     BrakeHoldTicks := 0;
652  ELSIF BrakeHold THEN
653
654     IF Tick THEN
655         BrakeHoldTicks := BrakeHoldTicks + 1;
656     END_IF;
657
658     (* v6v: signed web speed: a web pulled BACK (negative speed) also     *)
659     (* releases the T-one (trace 10/28 15:00: -6.6 m/min, about 20 cm).  *)
660
661     IF NOT ControlVoltage OR (Velocity_M_min < 5.0) OR (BrakeHoldTicks >= 150)
  THEN
662         BrakeHold := FALSE;
663     END_IF;
664  END_IF;
665
666 (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
667 (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
```

```
(* HMI: Set RwTeachCore (button, Set ON). Conditions: motor stopped, web *)
(* not moving, DT35 OK. After 2 s stable (filter settled):              *)
(*    RwAxisDist := RwDist + RwCoreDiameter / 2   -> RwDiam = RwCoreDiameter
 *)
(* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,              *)
(*               3 refused (motor running / web moving / DT35 fault).   *)
(* Only the offset is taught: the scale is RwDist10V - RwDist0V         *)
(* (= real distance between the two DT35 taught points).                *)

IF RwTeachCore THEN

   IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
     OR NOT RwFiltInit OR (RwCoreDiameter <= 0.0) THEN
      RwTeachState := 3;                    (* refused *)
      RwTeachCore := FALSE;
      TeachTicks := 0;
   ELSE
      RwTeachState := 1;                    (* teaching *)

      IF Tick THEN
         TeachTicks := TeachTicks + 1;
      END_IF;

      IF TeachTicks >= 20 THEN              (* 2 s stable *)
         RwAxisDist_Prev := RwAxisDist;     (* v6l: keep previous *)
         RwAxisDist := RwDist + RwCoreDiameter / 2.0;
         RwTeachState := 2;                 (* done *)
         RwTeachCore := FALSE;
         TeachTicks := 0;
      END_IF;
   END_IF;
ELSE
   TeachTicks := 0;

   IF O_MotorStart THEN
      RwTeachState := 0;                    (* clear status on next run *)
   END_IF;
END_IF;

(* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
(* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
(* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core   *)
(* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2          *)
(*                      + UwCoreDiameter   (General rung 14, unchanged). *)
(* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.               *)

IF Core_Measure THEN

   IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
     OR (UwDiam_Measure <= 0.0) THEN
      UwTeachState := 3;                    (* refused *)
      Core_Measure := FALSE;
      UwTeachTicks := 0;
      UwTeachSum := 0.0;
   ELSE
      UwTeachState := 1;                    (* teaching *)

      IF Tick THEN
         UwTeachTicks := UwTeachTicks + 1;
         UwTeachSum := UwTeachSum + UwDiam_Measure;
```

```
725             UwTeachSum := UwTeachSum + UwDiam_Measure;
726         END_IF;
727
728         IF UwTeachTicks >= 20 THEN              (* 2 s, 20 samples *)
729             UwCoreDist_Prev := UwCoreDistance;      (* v6l: keep previous *)
730             UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
731             UwTeachState := 2;                  (* done *)
732             Core_Measure := FALSE;
733             UwTeachTicks := 0;
734             UwTeachSum := 0.0;
735         END_IF;
736     END_IF;
737 ELSE
738     UwTeachTicks := 0;
739     UwTeachSum := 0.0;
740
741     IF O_MotorStart THEN
742         UwTeachState := 0;                      (* clear status on next run *)
743     END_IF;
744 END_IF;
745
746 (* ---- 14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
747 (* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
748
749 IF RwTeachUndo THEN
750
751     IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
752         TeachSwap := RwAxisDist;
753         RwAxisDist := RwAxisDist_Prev;
754         RwAxisDist_Prev := TeachSwap;
755         RwTeachState := 4;                      (* restored *)
756     ELSE
757         RwTeachState := 3;                      (* refused *)
758     END_IF;
759     RwTeachUndo := FALSE;
760 END_IF;
761
762 IF UwTeachUndo THEN
763
764     IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
765         TeachSwap := UwCoreDistance;
766         UwCoreDistance := UwCoreDist_Prev;
767         UwCoreDist_Prev := TeachSwap;
768         UwTeachState := 4;                      (* restored *)
769     ELSE
770         UwTeachState := 3;                      (* refused *)
771     END_IF;
772     UwTeachUndo := FALSE;
773 END_IF;
```

1-8-1-1-9.Tension_Manager

0 Tension_Manager.

```
1
2   //   x JOG boost while threading (JOG ... START), rate-limited,
3   //   1..10 V = TensMin_kg..TensMax_kg (T-one F.38 / F.39)
4   // TensAOEnable = FALSE -> AO Ch1 = fixed 10 V (old Fix10Vdc, pots supply)
5
6   // ---- 1. Paper type: KRAFT / BLEACHED selects the factor (N/mm width per 100 g/
    m2)
7
8   IF (TensFactorBleached < REAL#0.1) OR (TensFactorBleached > REAL#2.0) THEN
9       TensFactorBleached := REAL#0.54;
10  END_IF;
11
12  IF (TensFactorKraft < REAL#0.1) OR (TensFactorKraft > REAL#2.0) THEN
13      TensFactorKraft := REAL#0.80;
14  END_IF;
15
16  IF PaperKraft THEN
17      TensFactor := TensFactorKraft;
18  ELSE
19      TensFactor := TensFactorBleached;
20  END_IF;
21
22  // ---- 2. Helper inputs (limits)
23
24  IF PaperGSM < REAL#10 THEN
25      PaperGSM := REAL#10;
26  ELSIF PaperGSM > REAL#500 THEN
27      PaperGSM := REAL#500;
28  END_IF;
29
30  IF WebWidth_mm < REAL#100 THEN
31      WebWidth_mm := REAL#100;
32  ELSIF WebWidth_mm > REAL#3000 THEN
33      WebWidth_mm := REAL#3000;
34  END_IF;
35
36  // ---- 3. Recommended tension (kg)
37
38  TensIdeal_N := PaperGSM / REAL#100 * WebWidth_mm * TensFactor;
39  TensIdeal_Disp := TensIdeal_N / REAL#9.81;
40
41  // ---- 4. Setup limits (maintenance popup)
42
43  IF TensTaper < REAL#0.0 THEN
44      TensTaper := REAL#0.0;
45  ELSIF TensTaper > REAL#60.0 THEN
46      TensTaper := REAL#60.0;
47  END_IF;
48
49  IF TensCoreBoost < REAL#0.0 THEN
50      TensCoreBoost := REAL#0.0;
51  ELSIF TensCoreBoost > REAL#50.0 THEN
52      TensCoreBoost := REAL#50.0;
53  END_IF;
54
55  IF TensJogBoost < REAL#0.0 THEN
```

```
56      TensJogBoost := REAL#0.0;
57  ELSIF TensJogBoost > REAL#100.0 THEN
58      TensJogBoost := REAL#100.0;
59  END_IF;
60
61  IF TensRate_kg_s < REAL#1.0 THEN
62      TensRate_kg_s := REAL#1.0;
63  ELSIF TensRate_kg_s > REAL#20.0 THEN
64      TensRate_kg_s := REAL#20.0;
65  END_IF;
66
67  IF TensMin_kg < REAL#0.0 THEN
68      TensMin_kg := REAL#0.0;
69  END_IF;
70
71  IF TensMax_kg < TensMin_kg + REAL#5.0 THEN
72      TensMax_kg := TensMin_kg + REAL#5.0;
73  END_IF;
74
75  // ---- 5. Operator trim (HMI up/down, +/-10 kg), back to 0 when the paper changes
76
77  IF TensTrim_kg < REAL#-10.0 THEN
78      TensTrim_kg := REAL#-10.0;
79  ELSIF TensTrim_kg > REAL#10.0 THEN
80      TensTrim_kg := REAL#10.0;
81  END_IF;
82
83  IF (PaperGSM <> TensGSMMem) OR (WebWidth_mm <> TensWidthMem) OR
    (PaperKraft <> TensKraftMem) THEN
84      TensTrim_kg := REAL#0.0;
85  END_IF;
86
87  TensGSMMem := PaperGSM;
88  TensWidthMem := WebWidth_mm;
89  TensKraftMem := PaperKraft;
90
91  // ---- 6. Rewinder diameter fraction 0 (core) .. 1 (max diameter)
92
93  IF (RwMaxDiam > RwCoreDiameter + REAL#50.0) AND NOT RwSensFault THEN
94      TensDiaFrac := (RwDiam - RwCoreDiameter) / (RwMaxDiam - RwCoreDiameter);
95  ELSE
96      TensDiaFrac := REAL#0.0;                    // DT35 fault: no taper
97  END_IF;
98
99  IF TensDiaFrac < REAL#0.0 THEN
100     TensDiaFrac := REAL#0.0;
101 ELSIF TensDiaFrac > REAL#1.0 THEN
102     TensDiaFrac := REAL#1.0;
103 END_IF;
104
105 // ---- 7. Profile preview (HMI): core / mid / max
106
107 TensAtCore := TensIdeal_Disp * (REAL#1.0 + TensCoreBoost / REAL#100.0);
108 TensAtMid  := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0 * REAL#0.5);
109 TensAtMax  := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0);
110
111 // ---- 7b. Threading / knife adjustment preset: JOG raises the tension, START
    resumes normal
112
```

```
113 IF JogRun THEN
114     TensJogLatch := TRUE;
115 END_IF;
116
117 IF StartPB_OK AND NOT StartBlocked THEN          // START accepted -> normal
    tension
118     TensJogLatch := FALSE;
119 END_IF;
120
121 // ---- 8. Target: profile (taper) + trim, or manual; x JOG boost while threading
122
123 IF TensAuto THEN
124     TensTarget := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0 * TensDiaFrac) +
    TensTrim_kg;
125 ELSE
126     TensTarget := TensManual_kg;
127 END_IF;
128
129 IF TensJogLatch THEN
130     TensTarget := TensTarget * (REAL#1.0 + TensJogBoost / REAL#100.0);
131 END_IF;
132
133 IF TensTarget < TensMin_kg THEN
134     TensTarget := TensMin_kg;
135 ELSIF TensTarget > TensMax_kg THEN
136     TensTarget := TensMax_kg;
137 END_IF;
138
139 // ---- 9. Rate limit (4 ms scan): smooth change of GSM / width / type / mode /
    trim / JOG boost
140
141 TensStep := TensRate_kg_s * REAL#0.004;
142
143 IF TensSet_kg < TensTarget - TensStep THEN
144     TensSet_kg := TensSet_kg + TensStep;
145 ELSIF TensSet_kg > TensTarget + TensStep THEN
146     TensSet_kg := TensSet_kg - TensStep;
147 ELSE
148     TensSet_kg := TensTarget;
149 END_IF;
150
151 // ---- 10. Analog output Ch1
152
153 IF TensAOEnable THEN
154     // 1..10 V (400..4000) = TensMin_kg..TensMax_kg ; 0 V = wire broken (below
    range)
155     TensAOReal := REAL#400.0 + REAL#3600.0 * (TensSet_kg - TensMin_kg) /
    (TensMax_kg - TensMin_kg);
156
157     IF TensAOReal < REAL#400.0 THEN
158         TensAOReal := REAL#400.0;
159     ELSIF TensAOReal > REAL#4000.0 THEN
160         TensAOReal := REAL#4000.0;
161     END_IF;
162
163     AO_TOne_SetPoint := REAL_TO_INT(TensAOReal);
164 ELSE
165     AO_TOne_SetPoint := INT#4000;              // fixed 10 V (pots supply, as Fix10Vdc)
166 END_IF;
```



1-9.Data

1-9-1.Data Types

Name	Type	Comment

1-9-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
AO_TOne_SetPoint	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Analog O to T-one Tension SetPoint [Kg]
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDecelK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1285.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
Language	DINT			True	False	Publish Only	Language
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			True	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high

WorkSpd_Max	REAL	500.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
CntReset Mode	INT	0		True	False	Publish Only	Metre counter auto reset: 0 manual, 1 with new parent roll, 2 after auto-stop (at next START)
LastRoll_m	REAL	0.0		True	False	Publish Only	Length of the last roll (m), saved at the auto reset
PaperKraft	BOOL	False		True	False	Publish Only	Paper type: TRUE = kraft, FALSE = bleached (tension factor)
TensFactor Bleached	REAL	0.54		True	False	Publish Only	Tension factor bleached paper (N/mm width per 100 g/m2)
TensFactor Kraft	REAL	0.80		True	False	Publish Only	Tension factor kraft paper (N/mm width per 100 g/m2), tune on machine
TensAuto	BOOL	False		True	False	Publish Only	T-one set point from the PLC profile (FALSE = TensManual_kg)
TensAOEnable	BOOL	False		True	False	Publish Only	AO Ch1 = T-one set point (FALSE = fixed 10 V as old Fix10Vdc)
TensManual_kg	REAL	40.0		True	False	Publish Only	T-one manual set point (kg) when TensAuto is OFF
TensTaper	REAL	30.0		True	False	Publish Only	Tension taper core -> max diameter (%), 0..60
TensCoreBoost	REAL	0.0		True	False	Publish Only	Extra tension at the core (%), 0..50
TensRate_kg_s	REAL	5.0		True	False	Publish Only	T-one set point max change (kg/s), 1..20
TensMin_kg	REAL	10.0		True	False	Publish Only	T-one set point at 1 V (= T-one F.38)
TensMax_kg	REAL	100.0		True	False	Publish Only	T-one set point at 10 V (= T-one F.39)
TensSet_k	REAL	0.0		False	False	Publish Only	Live T-one set point s

g							ent on AO Ch1 (kg)
TensAtCore	REAL	0.0		False	False	Publish Only	Profile preview: tension at the core (kg)
TensAtMid	REAL	0.0		False	False	Publish Only	Profile preview: tension at mid diameter (kg)
TensAtMax	REAL	0.0		False	False	Publish Only	Profile preview: tension at max diameter (kg)
UwRawMin	INT	10		True	False	Publish Only	Unwinder sonar raw min (below = sensor fault, alarm 17)
UwRawMax	INT	3990		True	False	Publish Only	Unwinder sonar raw max (above = sensor fault / no echo, alarm 17)
TensCheckEnable	BOOL	False		True	False	Publish Only	Tension check ON (HMI switch); OFF = start as before
TensWaitSpd	REAL	10.0		True	False	Publish Only	Speed held after START until tension established (m/min)
TensionOK	BOOL	False		False	False	Publish Only	Tension established (debounced T-one Alarm 0)
TensWait	BOOL	False		False	False	Publish Only	Waiting for tension (speed held at TensWaitSpd, HMI lamp)
SonarAsk	BOOL	False		False	False	Publish Only	V10 HMI popup: sonar is OFF, turn it on? (HMI NO resets it)
UwSlow_at_D_Req	BOOL	True		True	False	Publish Only	V7c: unwinder diameter slow-down follows the speed (FALSE = fixed UwSlowDown_Diam)
UwSlowDiamAct	REAL	0.0		False	False	Publish Only	V7c: unwinder slows down at this diameter (mm), HMI display
FinalStopDecel	REAL	500.0		True	False	Publish Only	V12: end-of-roll stop ramp, web decel (mm/s2), 0 = drive DEC ramp as before, 50..2000
FinalStopRamp	BOOL	False		False	False	Publish Only	V12: end-of-roll stop ramp running (HMI lamp)

RwJogMaxDiam	REAL	400.0		True	False	Publish Only	V7d: jog blocked above this rewinder diameter (mm), 0 = no limit
JogInhibit	BOOL	False		False	False	Publish Only	V7d: jog blocked (rewinder roll too big), HMI lamp
SCurveTime	REAL	0.5		True	False	Publish Only	V7f: S-curve rounding of the AUTO ramps (s), 0 = off, max 2
TOneDecel	BOOL	False		False	False	Publish Only	V7f: line decelerating (for a T-one stop/boost input)
HornStartBeep_s	REAL	1.0		True	False	Publish Only	V9 horn: beep when the slow-down starts (s), 0 = off, max 5
HornBeepLen_s	REAL	0.2		True	False	Publish Only	V9 horn: length of the short beeps during slow-down (s), 0 = off, max 2
HornBeepPeriod	REAL	5.0		True	False	Publish Only	V9 horn: one short beep every ... s during slow-down, 0 = off, max 30
HornStopBeep_s	REAL	3.0		True	False	Publish Only	V9 horn: beep when stopped after a slow-down (s), 0 = off, max 10
HornTest	BOOL	False		False	False	Publish Only	V9 horn: HMI test button (momentary), machine stopped only
RwLockDiam	REAL	300.0		True	False	Publish Only	V9: paper thickness LOCKED at this rewinder diameter (mm), 0 = never lock
RwLockMinGrow_mm	REAL	50.0		True	False	Publish Only	V9: min diameter growth since the start of learning before locking (mm)
RwMatLocked	BOOL	False		True	False	Publish Only	V9: paper thickness locked (HMI lamp), NEW PAPER / GSM YES unlocks
LongThick_mm	REAL	0.0		False	False	Publish Only	V9: thickness from the long baseline (mm), HMI display
RwDiamRev	REAL	0.0		False	False	Publish Only	V9: rewinder diameter averaged over one revolution (

							mm)
RwWave_mm	REAL	0.0		False	False	Publish Only	V9: diameter wave peak-peak in the last revolution (mm), HMI
UwStopLenShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder STOP LENGTH (AUTO STOP UNWINDER active)
UwSlowDiamShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder SLOW AT diameter (normal operation, sonar ON)
SoftCancelAccel	REAL	100.0		True	False	Publish Only	V9: then ramp up at this web accel (mm/s2), 0 = AccelWeb
RwMatReset	BOOL	False		False	False	Publish Only	HMI NEW PAPER: forget the learned paper thickness RwMat_K (PLC resets it)
FinalSpd_Max_Set	REAL	80.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	80.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwLearnReset	BOOL	False		False	False	Publish Only	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
RwCoreDiameter	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH CORE (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Percent	REAL	12.0		True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder P	False	False	Publish Only	Encoder counter

			resent Position				
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY [0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	In_0 ControlVoltageON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	In_1 Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	In_2 Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	In_3 Motor thermal switch

I_Prot_By Pass	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 04	False	False	Publish Only	In_4 Guards-bypassed selector switch - Under Panel
I_Hub_Po sition	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 05	False	False	Publish Only	In_5 Hub/chuck in extraction position
I_FrontGu ard_Close d	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 06	False	False	Publish Only	In_6 Front guard closed
I_Jog_PB	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 07	False	False	Publish Only	In_7 Jog Push button
I_VFD_Al arm	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 08	False	False	Publish Only	In_8 Inverter in alarm
I_Side_Do or_Closed	BOOL		BuiltInIO: //cpu/#0/I nput_Bit_ 13	False	False	Publish Only	In_Side Door is Closed
I_OR_Posi tion_Sens	BOOL		ECAT://no de# [1,0]/IB IL 24 DI 8/HD- ECO Input -_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Hoo ver_Mode	BOOL		ECAT://no de# [1,0]/IB IL 24 DI 8/HD- ECO Input -_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider Hoover Mode
I_OR_Aut o_OS	BOOL		ECAT://no de# [1,0]/IB IL 24 DI 8/HD- ECO Input -_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic

I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Q_0 Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Q_1 Motor ON lamp
O_RwUnload	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPa	BOOL		ECAT://node#[1,1]/I	False	False	Publish Only	Front guard bypass

ss			B IL 24 DO 8/HD-ECO Output_OUT04 (Pin 2.2) _7000_04				
O_Brake	BOOL		BuiltInIO: //cpu/#0/Output_Bit_02	False	False	Publish Only	Q_2 Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO: //cpu/#0/Output_Bit_06	False	False	Publish Only	Q_6 OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO: //cpu/#0/Output_Bit_09	False	False	Publish Only	Q_9 Raise OverRider ~10mm Over Round Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO: //cpu/#0/Output_Bit_04	False	False	Publish Only	Q_4 OverRider Lock
O_Cooling Fan	BOOL		BuiltInIO: //cpu/#0/Output_Bit_03	False	False	Publish Only	Q_3 Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message: Warnings / Alarms
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO: //cpu/#0/Input_Bit_09	False	False	Publish Only	In_9 Front guard open
O_RwLoading_Enab	BOOL		BuiltInIO: //cpu/#0/O	False	False	Publish Only	Q_7 Unloaders Load enable - closing

le			utput_Bit_07				
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Q_5 Second acc / dec ramp
AccelTrim	REAL	0.08		True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL	0.08		True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL	0.3		True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL	0.3		True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
UwStopDiamShow	BOOL	False		False	False	Publish Only	V11 HMI: show unwinder STOP AT diameter (slow-down reached, decel / final speed)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Q_8 Horn / Acoustic signal
Pot2_Slow_Velocity	INT		BuiltInIO://opt/#0/Ch2_Analog_Input_Value	False	False	Publish Only	Potentiometer 2 value - Final speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	In_10 Stop on metre counter
I_Input_11_Spare_1255	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Do not publish	In_11 SPARE (wire 12.55) Under Panel
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	In_12 Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input	False	False	Publish Only	Extractor fan thermal OK

			Status/Ch 1 External Input 0 Status				
I_T_one_TensionOK	BOOL		ECAT://node# [1,0]/IB IL 24 DI 8/HD-ECO Input -_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	T-one Alarm 0 (15-16): tension established (replaces I_SlowSens_PH)
I_WebBreak_PH	BOOL		ECAT://node# [1,0]/IB IL 24 DI 8/HD-ECO Input -_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_at_M_Val	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1315.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDecelLrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			False	False	Publish Only	Password Entry
I_RsetAlarm	BOOL		ECAT://node# [1,0]/IB IL 24 DI 8/HD-ECO Input -_IN08 (Pin 2.4)_	False	False	Publish Only	Alarm reset

			6000_08				
EncoderWheel_Circumf	REAL	402		True	False	Publish Only	Encoder wheel (Cylinder / Roller) Circumference [mm]
AlarmCodeSV	ARRAY [0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	8.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Unwinder Diameter measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlow_at_D_Val	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwStopDiam_Limit	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	Password 22 for Popup 22
Psw44	BOOL			False	False	Publish Only	Password 44 for Popup 44
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2	False	False	Publish Only	Analog value - ReWinder Diameter

			Analog Input Value				
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	50.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	782.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	727.0		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	15.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.03		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm

							)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]
RwStop_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlow_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwinder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_M_Val	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
UwSlow_at_M_Req	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStop_at_D_Val	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	30.0		True	False	Publish Only	Unwinder Empty Hysteresis
GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
RwMaxSlowAtDiam	REAL	1370.0		False	False	Publish Only	Rewinder max: slow-down starts at diamete

							r (mm)
RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
SoftStopDecel	REAL	500.0		True	False	Publish Only	Soft stop decel (mm/s2), 0 = DecelCoarse
SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL	727.0		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)
Psw33	BOOL			False	False	Publish Only	Password 33 for Popup 33
RwMatChangePct	REAL	35.0		True	False	Publish Only	Paper change detection: window deviation (%), 2 windows in a row = new paper, 0 = off
RwSlowFallbackMm	REAL	30.0		True	False	Publish Only	Rewinder: slow-down this many mm before the target while the paper thickness is not learned (0 = off)
RwAutoMaxSpd	REAL	500.0		True	False	Publish Only	Rewinder AUTO stop ON: maximum work speed (m/min), 0 = no limit
RwWinLen_m	REAL	60.0		True	False	Publish Only	Paper thickness learning: MAX window length (m of paper)
RwWinGrow_mm	REAL	5		True	False	Publish Only	Paper thickness learning: window = this diameter growth (mm), 4 m .. RwWinLen_m

GsmAsk	BOOL	False		False	False	Publish Only	HMI popup: new parent roll, did you change the paper? (HMI NO resets it)
GsmAskYes	BOOL	False		False	False	Publish Only	HMI popup YES: forget the learned thickness (PLC resets it)
DecelWeb	REAL	250.0		True	False	Publish Only	V7: AUTO decel ramp, real web mm/s2 (0 = DecelCoarse)
AccelWeb	REAL	200.0		True	False	Publish Only	AUTO accel ramp, real web mm/s2 (0 = AccelCoarse)
RwRatioD	REAL	3366.0		True	False	Publish Only	V7: drive counts per m/min x rewinder diameter (mm), adapted while running
FFEnable	BOOL	True		True	False	Publish Only	V7: feed-forward speed loop ON (FALSE = old loop of rung 16)
SpdHold	BOOL	False		False	False	Publish Only	V9: speed held after START during a soft stop, START again = ramp up (HMI lamp)
SonarAskYes	BOOL	False		False	False	Publish Only	V10 HMI popup YES: sonar ON (PLC resets it)
UwStop_at_D_Val	REAL	0.0		False	False	Publish Only	V11: unwinder stop command diameter (mm) so the web stops at UwDiamStop, HMI "STOP AT"
Clock10ms	BOOL			False	False	Publish Only	10mS clock
Clock100ms	bool			False	False	Publish Only	100mS clock
Sec_to_Min	REAL			False	False	Do not publish	
Psw48	BOOL			False	False	Publish Only	Password 48 for Popup 48
JogRun	BOOL			False	False	Publish Only	V10: JOG really running (button, not a normal run, not a soft stop)
GuardStopOK	BOOL			False	False	Publish Only	V11: stopped for real -> guard bypass allowed (Output 13)
SlowDow	BOOL			False	False	Publish Only	Slow down

n							
WarningOn	bool			False	False	Publish Only	HMI Warning Display
AlarmSilenced	BOOL			False	False	Publish Only	
PaperGSM	REAL			True	False	Publish Only	
WebWidth_mm	REAL			True	False	Publish Only	
TensFactor	REAL			True	False	Publish Only	Tension Helper
TensIdeal_Disp	REAL			True	False	Publish Only	Tension Helper
TensLostWarn	BOOL	False		False	False	Publish Only	Tension lost while running (warning)
UwSensTest	BOOL	False		False	False	Publish Only	Unwinder sonar fault (raw out of range 1 s)
UwSensFault	BOOL			False	False	Publish Only	UnWinder Sensor Fault
Thickness_Fault	BOOL			False	False	Publish Only	Thickness Not Learned !
AutoStopWhy	INT			False	False	Publish Only	
AutoStopDone	BOOL			False	False	Publish Only	
AutoEnRefused	BOOL			False	False	Publish Only	
AutoEnWhy	INT			False	False	Publish Only	
TensWaitHz	REAL	1.0		True	False	Publish Only	
StartBlocked	BOOL	True		False	False	Publish Only	
GuardOpen	BOOL	False		False	False	Publish Only	
SideDoorOpen	BOOL	False		False	False	Publish Only	
TensDiaFrac	REAL	0.0		False	False	Publish Only	Rewinder diameter fraction 0 (core) .. 1 (max)
Psw40	BOOL			False	False	Publish Only	Password 40 for Popup 40
TensJogBoost	REAL	30.0		True	False	Publish Only	Threading preset: % extra tension from JOG until START, 0..100
TensJogLatch	BOOL	False		False	False	Publish Only	Threading tension active (JOG -> START), HMI lamp
TensTrim_kg	REAL	0.0		True	False	Publish Only	Operator trim on the AUTO tension (kg), +

								/-10, reset when paper changes
	CntAutoReset	BOOL			True	False	Publish Only	Metres Counter Auto Reset with Choices
	CntResetPend	BOOL	False		True	False	Publish Only	Metre counter reset waiting for the next START (HMI lamp)
	RwSlowLearnMin_mm	REAL	50.0		True	False	Publish Only	Slow-down learning: min decel accepted (mm/s2), below = ignored